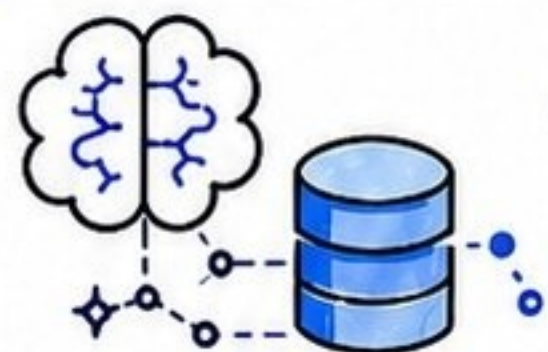




RAG ROADMAP

COMPLETE SYLLABUS



1 RAG Fundamentals

- What is RAG?
- Traditional RAG vs Agentic RAG
- RAG architecture & workflow
- RAG limitations

2 Document Ingestion

- Document loaders
- Parsing & extraction
- Cleaning & normalization
- Metadata extraction
- OCR for scanned documents

3 Chunking

- Fixed-size chunking
- Recursive chunking
- Semantic chunking
- Parent-child chunking
- Metadata-aware chunking

4 Embeddings

- Embedding models
- Dense vs sparse embeddings
- Similarity metrics
- Multilingual embeddings
- Embedding evaluation

5 Indexing & Vector Databases

- Vector stores
- HNSW / ANN
- Metadata filtering
- Hybrid indexing
- Popular vector DBs: FAISS, Pinecone, Weaviate, Milvus, Qdrant

6 Retrieval

- Semantic search
- Keyword/BM25 search
- Hybrid search
- Query expansion
- Multi-query retrieval
- Self/Corrective retrieval

7 Reranking

- Cross-encoder reranking
- LLM reranking
- Top-K → Top-N retrieval
- Retrieval optimization

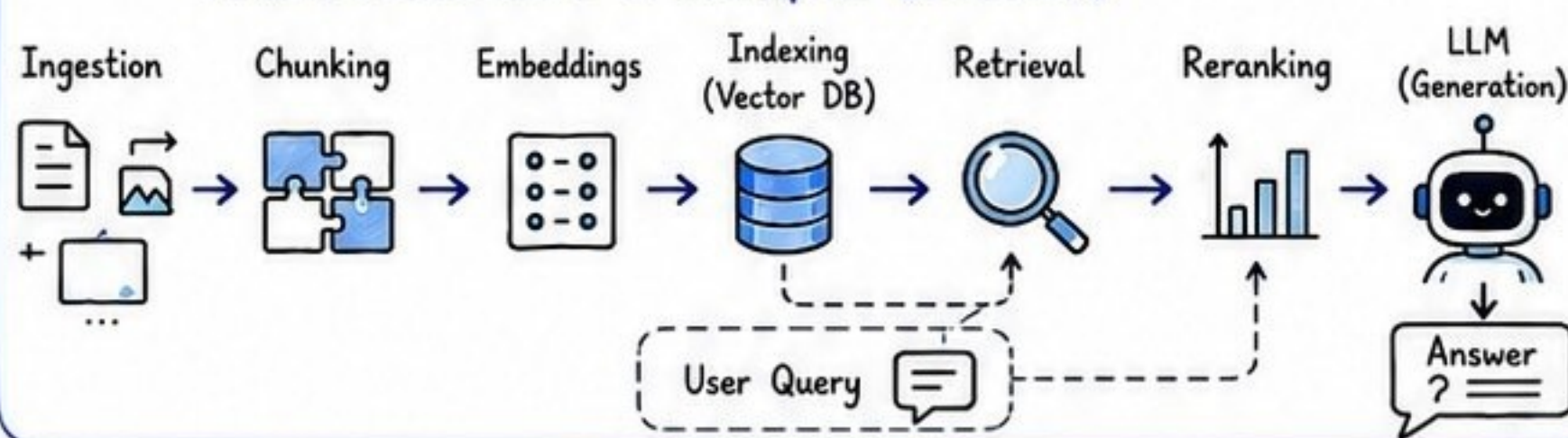
8 RAG Evaluation

- Retrieval evaluation
- Precision / Recall / MRR / NDCG
- Faithfulness
- Answer relevance
- Context relevance
- RAGAS / DeepEval
- End-to-end evaluation

9 Advanced Topics

- Graph RAG
- Knowledge Graph + RAG
- SQL + RAG
- Multimodal RAG
- Hybrid RAG
- Corrective RAG (CRAG)
- Self-RAG
- Adaptive RAG
- Deep Research Agents

RAG Architecture & Workflow (Overview)



RAG Limitations

- Dependent on data quality
- Retrieval may miss relevant info
- Hallucination can still occur
- Context window limitations
- Complex to evaluate & deploy
- Higher latency with more steps



Better Answers
with Your Data



Up-to-date
& Reliable



Scalable &
Flexible



Enterprise
Ready



Learn Once,
Apply Everywhere

☆ RAG Fundamentals ☆

① What is RAG?

RAG (Retrieval Augmented Generation) is a technique that combines information retrieval from external knowledge sources, with LLM generation to produce accurate, up-to-date and context-aware answers.

Key Idea

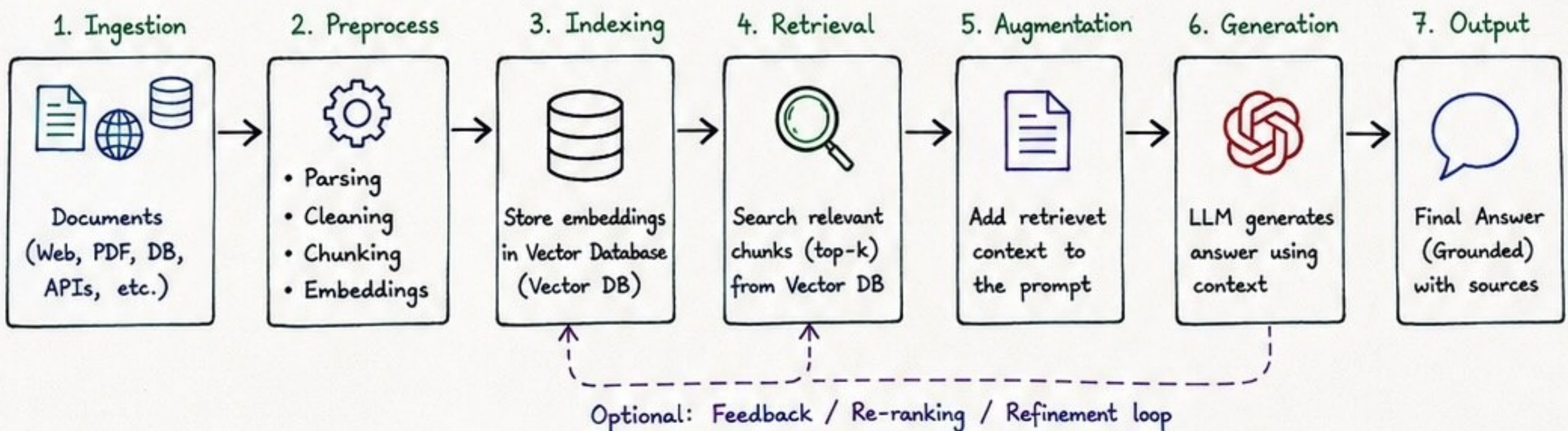
Instead of relying only on the LLM's pre-trained knowledge, RAG retrieves relevant information at query time and uses it as context to generate better answers.



② Traditional RAG vs Agentic RAG

Aspect	Traditional RAG	Agentic RAG
Approach	Single-step retrieve then generate	Agent plans, decides and acts using tools
Retrieval	One-shot retrieval (top-k)	Iterative / multi-step retrieval, dynamic queries
Reasoning	LLM reasons only on retrieved context	Agent reasons, reflects and refines
Control	Static pipeline	Agent-driven workflow
Workflow	Retrieve → Augment → Generate	Plan → Search → Analyze → Refine → Generate
Example	Chat with docs, simple Q&A	Research agent, deep analysis, multi-hop QA

③ RAG architecture & workflow



Typical Data Flow:

1. User asks a question.
2. System retrieves relevant info from knowledge source.
3. Retrieved context is provided to LLM.
4. LLM generates answer.
5. System returns grounded answer with sources.

④ RAG limitations

- Quality depends on retrieval - If relevant info is not retrieved, answer will be poor.
- Retrieval misses - Irrelevant or incomplete context can lead to wrong answers.
- Context window limits - Large documents may not fit in LLM context.
- Latency & cost - Retrieval + generation adds extra time and cost.
- Complex reasoning - Multi-hop or highly complex queries may be hard to answer.
- Hallucination risk - LLM may still hallucinate if context is weak or ambiguous.
- Maintenance overhead - Index updates, chunking strategy, embeddings need regular care.

Bottom line

RAG improves accuracy and freshness by using external knowledge, but it has limitations in retrieval quality, cost, latency and complex reasoning. ☆

Quick Takeaway ☆

RAG = Retrieve relevant information → Augment with context → Generate grounded answer.
Agentic RAG takes it further with planning, reasoning and iterative retrieval.



☆ Document Ingestion ☆

Document ingestion is the first step in any RAG pipeline. It involves taking documents from various sources and preparing them so they can be stored, searched and used by LLMs.

1. Document loaders

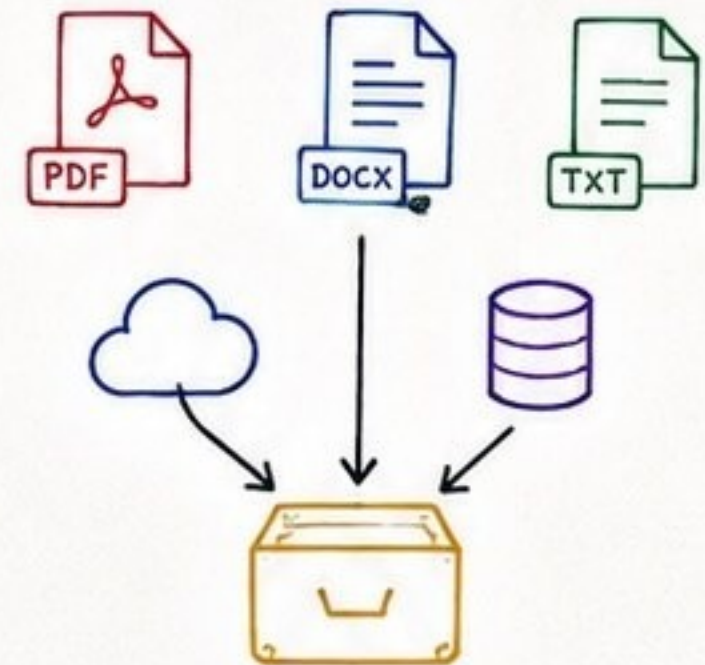
Load documents from different sources into the system.

Common sources:

- Local files (PDF, TXT, DOCX, CSV)
- Web pages (URLs)
- Databases (SQL/NoSQL)
- Cloud storage (S3, GCS, Azure Blob)
- APIs and other integrations

Example loaders:

- PyPDFLoader (PDF)
- TextLoader (TXT)
- Docx2txtLoader (DOCX)
- CSVLoader (CSV)
- UnstructuredURLLoader (Web)
- SQLDatabaseLoader (Database)
- S3Loader / GCSLoader

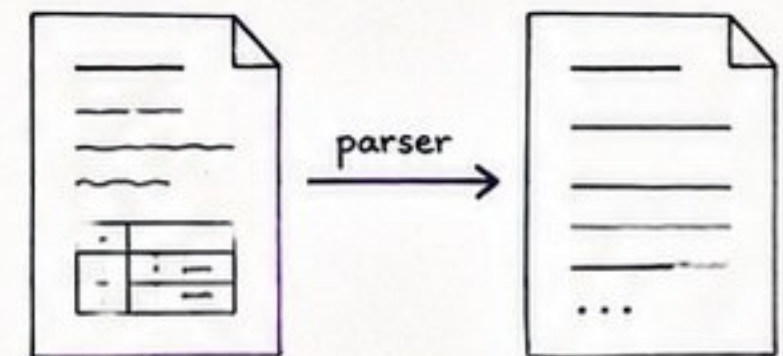


2. Parsing & extraction

Convert raw documents into structured text. Extract the meaningful text along with structure information.

What we extract?

- Plain text
- Headings / Titles
- Paragraphs
- Tables
- Lists
- Images (if needed)
- Page numbers, sections etc.



3. Cleaning & normalization

Clean and standardize the extracted text to improve quality and consistency.

Common steps:

- Remove extra spaces, line breaks
- Remove headers/footers, page numbers
- Fix encoding issues
- Remove special characters / noise
- Normalize unicode, formats, dates, numbers
- Expand abbreviations (if needed)

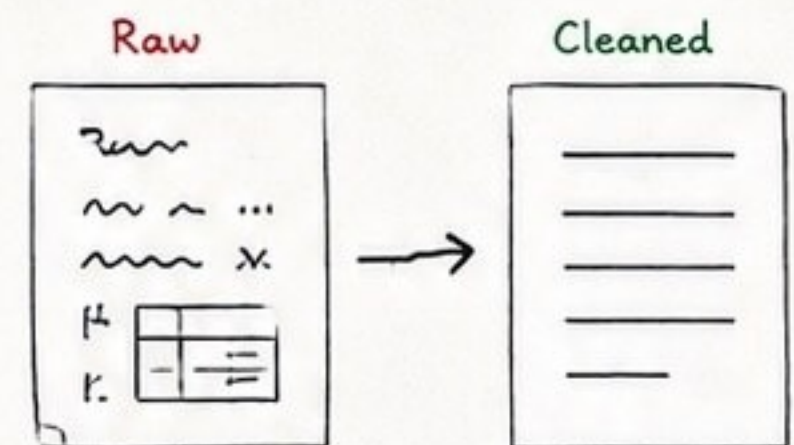
Example:

Raw Text:

" This is an
Example!!! -- It has
Random spaces,
Newlines."

Cleaned Text:

This is an Example! It has
random spaces and newlines.



4. Metadata extraction

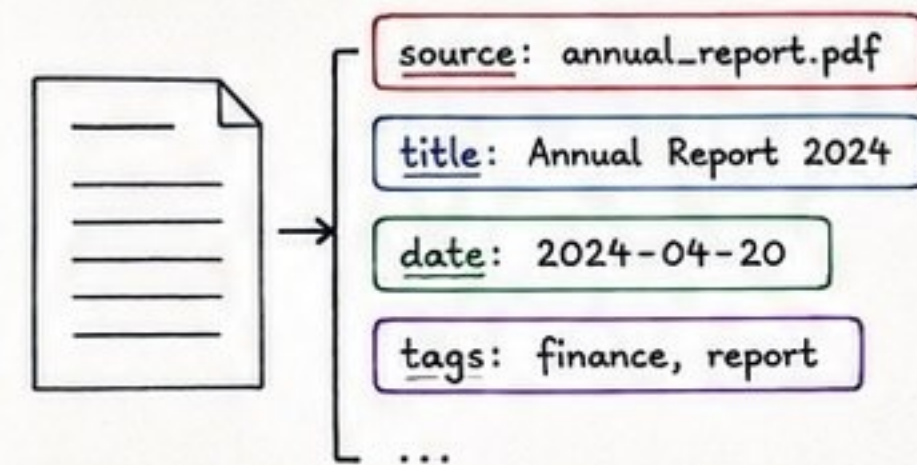
Extract useful metadata to improve searching, filtering and access control.

Common metadata:

- Source (file name, URL, table name)
- Title / Heading
- Author / Publisher
- Date / Created time
- Document type
- Page number / Section
- Tags / Categories / Custom fields

Example metadata:

```
{
  "source": "annual_report.pdf",
  "title": "Annual Report 2024",
  "author": "ABC Corp",
  "date": "2024-04-20",
  "page": 12,
  "section": "Financial Performance",
  "tags": ["finance", "report"]
}
```



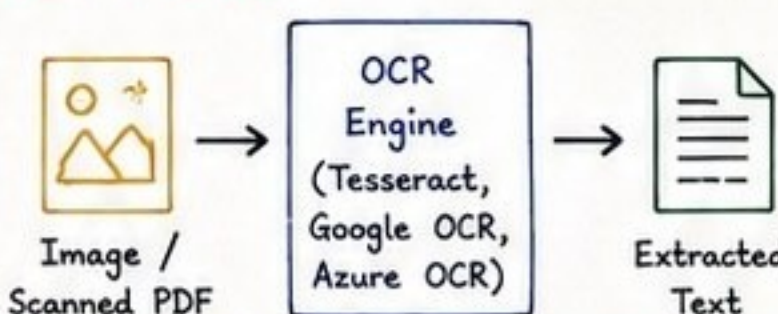
5. OCR for scanned documents

Use Optical Character Recognition (OCR) to extract text from scanned images or PDFs.

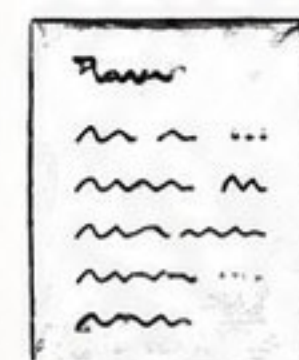
When to use OCR?

- Scanned PDFs
- Images (JPG, PNG, TIFF)
- Handwritten or printed documents

How it works?



Scanned Document



Extracted Text



Good ingestion = Better retrieval = Better answers from LLMs!



★ Chunking ★

Chunking is the process of splitting documents into smaller pieces (chunks) so they can be embedded and retrieved effectively.

① Fixed-size chunking

Split documents into chunks of fixed number of characters or tokens with optional overlap.

Example:

Chunk size = 100 tokens
Overlap = 20 tokens

Document

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.

Chunk 1 (0-100)

Lorem ipsum dolor sit amet, consectetur adipiscing elit.

Chunk 2 (100-200)

Sed do eiusmod tempor incididunt ut labore et

Chunk 3 (200-300)

dolore magna aliqua. Ut enim ad minim veniam, quis

Use when:

- Simple & fast
- Works well for uniform text
- Good baseline for most applications



② Recursive chunking

Recursively split text by logical separators (e.g., headings, paragraphs, sentences) until chunks fit the size limit.

Example:

Max size = 200 tokens
Separators = ["\n\n", "\n", ".", " "]

Document (with headings)

1. Introduction
This is the intro...
It has multiple sentences.
2. Methodology
This section explains...
It has more details...
2.1 Data Collection
We collected data by...
2.2 Preprocessing
We cleaned the data by...

Chunk 1: Introduction

This is the intro...
It has multiple sentences.

Chunk 2: Methodology

This section explains...
It has more details...

Chunk 3: Data Collection

We collected data by...

Use when:

- Preserve structure
- Better for documents with headings and sections
- More meaningful chunks



③ Semantic chunking

Split text based on semantic similarity. Create breaks where meaning shifts significantly.

Example:

Break when similarity score < threshold

Document

Topic A (related sentences)
Topic A
Topic B (different idea)
Topic B
Topic C (new idea)
...

Chunk 1 (Topic A)

related sentences...

Chunk 2 (Topic B)

different idea...

Chunk 3 (Topic C)

new idea...

Use when:

- Keep semantically coherent chunks
- Improves retrieval quality
- Needs embeddings to detect boundaries



④ Parent-child chunking

Split into small child chunks for retrieval, but keep a larger parent chunk for more context.

Example:

Child size = 150 tokens
Parent size = 800 tokens

Parent Chunk (Large)

Section: 2. Methodology
(Full content of the section...)

Child 1

Text part 1...

Child 2

Text part 2...

Child 3

Text part 3...

How it works?

1. Retrieve top-k child chunks
2. Return their parent chunk to LLM for more context

Use when:

- Need fine-grained retrieval
- But want more context in answers
- Improves relevance and coherence



⑤ Metadata-aware chunking

Create chunks while considering metadata like title, section, source, page number, date, author, etc.

Example:

Attach metadata to each chunk during creation.

Document

Title: Annual Report 2024
Section: Financial Performance
Page: 12
...
(Actual content of the page/section...)

Chunk 1

Content...
{title: Annual Report 2024, section: Financial Performance, page: 12}

Chunk 2

Content...
{title: Annual Report 2024, section: Financial Performance, page: 13}

Use when:

- Need better filtering & access control
- Important for enterprise data
- Improves traceability and relevance



Good chunking = Better retrieval → Better context → Better answers from LLMs!



Embedding (Part 1)



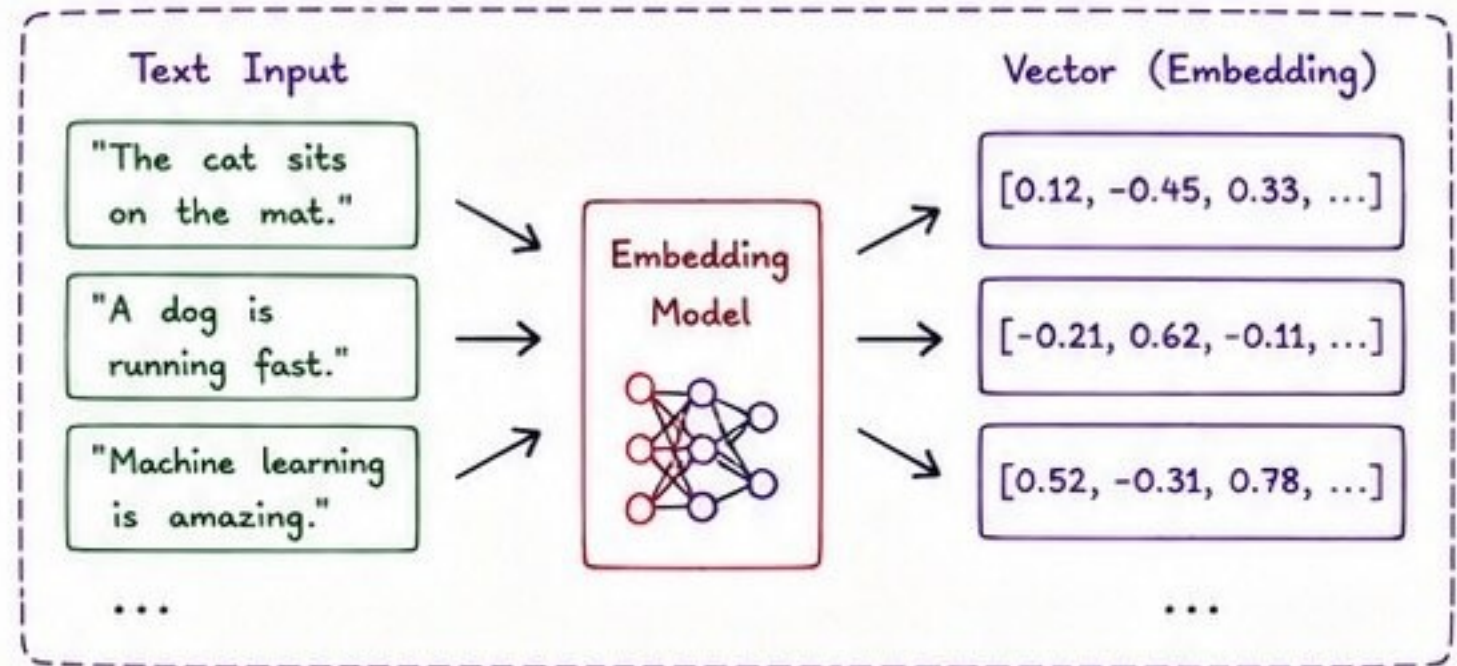
Embeddings are numerical representations of text, documents or other data in a vector space so that machines can understand and compare meaning.

1. Embedding models

Embedding models convert text (words, sentences, documents) into vectors of numbers (embeddings). These vectors capture semantic meaning.

Popular embedding models:

- OpenAI: text-embedding-3-small, text-embedding-3-large
- Hugging Face: all-MiniLM-L6-v2, BAAI/bge-large-en-v1.5
- Cohere: embed-english-v3.0
- Sentence Transformers: all-mpnet-base-v2
- Others: E5, GTR, Instructor, SBERT, etc.



High quality embeddings = similar meanings are closer in vector space.

2. Dense vs Sparse embeddings

	Dense Embeddings	Sparse Embeddings
Definition	Vectors are dense, most values are non-zero.	Vectors are mostly zeros, few non-zero values.
Representation	High-dimensional, continuous values. Example: [0.23, -0.11, 0.67, ..., 0.05]	High-dimensional, but mostly zeros. Example: [0, 0, 0.56, 0, 0, ..., 0]
Example Models	OpenAI, BERT, MiniLM, SBERT, Cohere	TF-IDF, BM25, SPLADE, UniCOIL
Advantages	• Capture semantic meaning well • Good for similarity search • Handle synonyms & context	• Fast and memory efficient • Better for exact term matching • Works well with keyword search
Disadvantages	• Higher computation & storage • Needs more data to train	• Poor at capturing semantics • No understanding of context/synonyms
Use Cases	Semantic search, recommendations, clustering, RAG retrieval	Keyword search, filtering, information retrieval, BM25 ranking
Analogy	 Like a photo (captures overall meaning & context).	 Like a bag of words (counts important words only).

Key Takeaway

Dense = better for meaning (Semantic search)

Sparse = better for exact match (Keyword search)

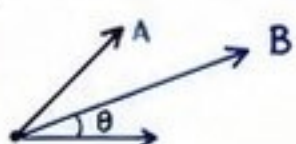
Hybrid (Dense + Sparse) gives the best results.

3. Similarity metrics

Similarity metrics measure how close two vectors are in the vector space.

1. Cosine Similarity

Most commonly used. Measures the cosine of the angle between two vectors.

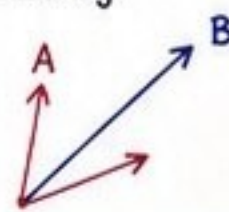


$$\cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}$$

Range: [-1, 1] (1 is most similar)

2. Dot Product

Measures the magnitude of similarity.



$$A \cdot B = \sum_{i=1}^n A_i \times B_i$$

Higher score = more similar (Not normalized)

3. Euclidean Distance

Measures the straight-line distance between vectors.

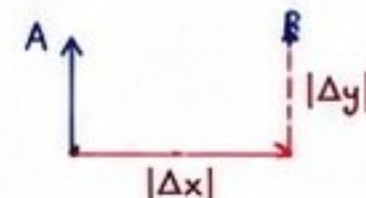


$$d(A, B) = \sqrt{\sum_{i=1}^n (A_i - B_i)^2}$$

Lower distance = more similar

4. Manhattan Distance (L1 Distance)

Sum of absolute differences between vector components.



$$d(A, B) = \sum_{i=1}^n |A_i - B_i|$$

Lower distance = more similar

5. Dot Product (Similarity)

Useful in some retrieval systems.

$$\text{sim}(A, B) = A \cdot B$$

Higher = more similar



In practice: Cosine Similarity is most popular for dense embeddings.

BM25 / TF-IDF (sparse) uses different scoring based on term frequency & inverse document frequency.





Embedding (Part 2)



1. Multilingual embeddings

Multilingual embeddings map words/sentences from different languages into the same vector space, so that semantically similar text across languages are close to each other.

Why needed?

- Enable cross-language search
- Support global applications
- Handle multiple languages in the same system
- Improve inclusivity and user experience

Example

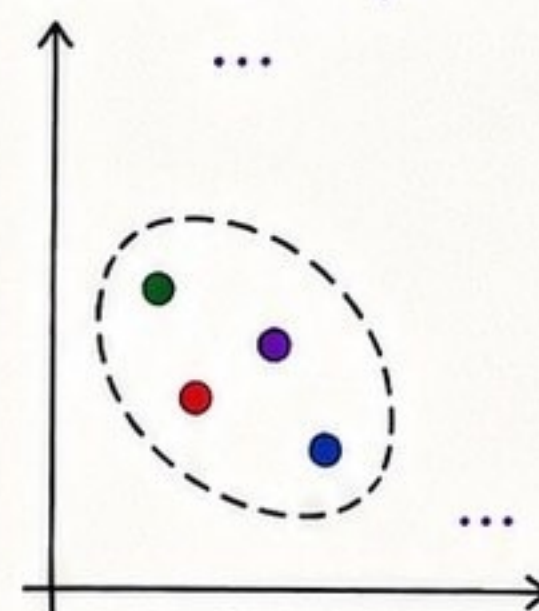
English: "Artificial intelligence"

Spanish: "Inteligencia artificial"

Hindi: "कृत्रिम बुद्धिमत्ता"

Chinese: "人工智能"

Shared Vector Space



Popular multilingual embedding models

- LaBSE (Google)
- mUSE (Facebook)
- multilingual-e5 (intfloat)
- BGE-M3 (BAAI)
- Cohere Embed Multilingual
- OpenAI text-embedding-3-large (supports many languages)

Key points ☆

- Use models trained on parallel / multi-language data.
- Performance depends on data quality & language coverage.
- Some low-resource languages may still have lower quality.
- Normalize text (lowercase, unicode) before embedding.
- Evaluate on cross-lingual tasks, not just monolingual.

2. Embedding evaluation

Evaluate embeddings to ensure they capture semantic meaning well and work for the target use case.

What to evaluate?

- Semantic similarity (are similar texts close?)
- Retrieval quality (do they help find right results?)
- Clustering quality (do similar items group together?)
- Downstream task performance
- Robustness (noise, typos, domain shift, cross-lingual)

Common evaluation datasets

- STS (Semantic Textual Similarity)
- MTEB (Massive Text Embedding Benchmark)
- BEIR (Benchmark for IR)
- MIRACL (Multilingual retrieval benchmark)
- Custom domain datasets

Key evaluation methods & metrics

Goal	Method	Metrics	What it measures
Semantic similarity	STS task	Pearson, Spearman	Correlation with human similarity scores
Retrieval quality	Information Retrieval	Recall@k, MRR, NDCG@k, MAP	How well embeddings retrieve relevant documents
Clustering quality	Clustering	Silhouette Score, NMI, ARI	How well similar items are grouped
Downstream tasks	Classification / QA / RAG	Accuracy, F1, EM	Impact on real tasks
Efficiency	Benchmark / Inference	Throughput, Latency, Memory	Cost of using the embeddings



Bottom line:

Good embeddings = High quality retrieval + Better downstream performance + Lower cost.





Indexing & Vector Databases (Part 1)



Indexing stores embeddings in a vector database so they can be searched quickly and efficiently at scale.

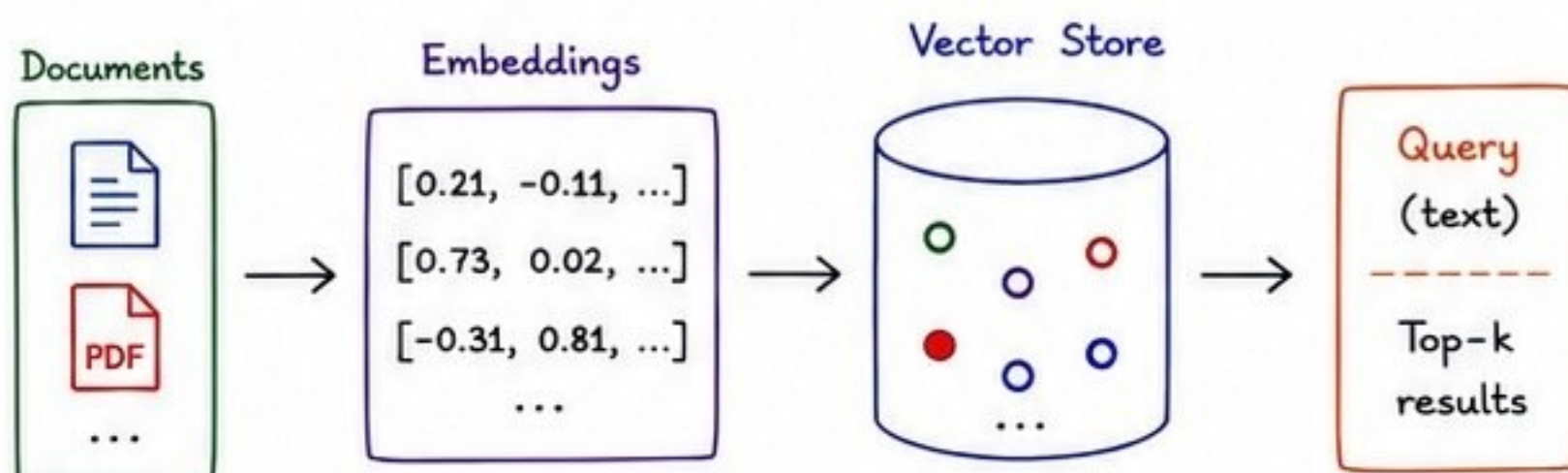
1. Vector stores

Vector stores (or vector databases) are purpose-built databases that store high-dimensional vectors and enable similarity search.

Key features

- Store embeddings + optional metadata
- Fast similarity search
- Scalable to millions/billions of vectors
- Support filtering, upserts, deletes
- Persist data and provide APIs

How it works?



Popular vector stores

Vector Store	Type	Notes
Pinecone	Managed (Cloud)	Fully managed, scalable, serverless
Weaviate	Open Source / Cloud	Hybrid search, modules, GraphQL
Milvus	Open Source	High performance, scalable, multi-modal
Qdrant	Open Source	Fast, payload filtering, easy to use
FAISS	Open Source (Library)	In-memory, high performance (Meta)
Chroma	Open Source	Simple, lightweight, great for dev

2. HNSW / ANN

Exact search is slow for large datasets. We use Approximate Nearest Neighbor (ANN) algorithms to get very close results much faster.

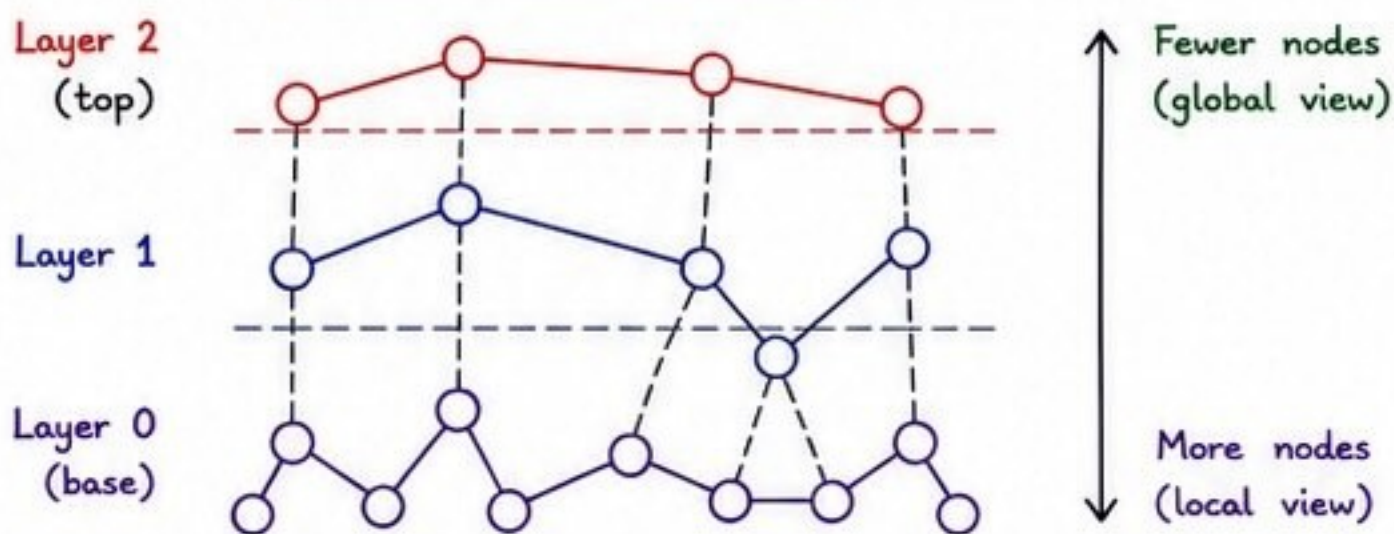
HNSW (Hierarchical Navigable Small World)

- Graph-based ANN algorithm
- Builds multi-layer proximity graphs
- Search starts from top layer → navigates down to bottom layer
- Very fast and high recall
- Widely used in modern vector DBs

ANN Search Flow (HNSW)

1. Start from entry point at top layer
2. Find nearest neighbors in that layer
3. Move down a layer
4. Repeat until base layer
5. Return top-k nearest neighbors

HNSW Structure (Example)



Why ANN?

- Exact search $\sim O(N)$ → too slow for large N
- ANN $\sim O(\log N)$ or sub-linear → very fast
- Slightly approximate but good enough for RAG

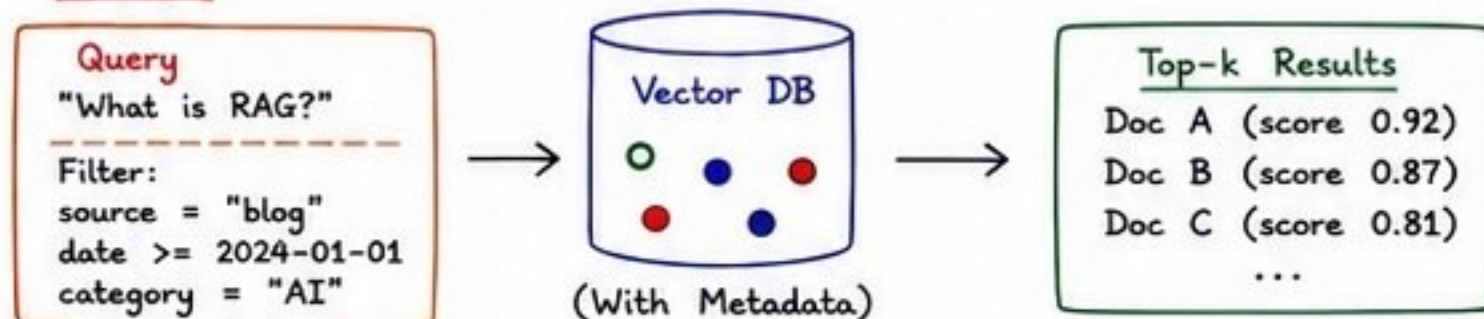
3. Metadata filtering

Metadata filtering narrows down the search space using structured fields (e.g., source, date, author, category, permissions).

Why use it?

- Improve relevance
- Enforce access control
- Reduce search space → faster & cheaper
- Support multi-tenant / domain-specific search

Example



Common metadata fields

source / file_name	page / chunk_id	author / created_by	date / timestamp
category / tags	language	permissions / ACL	custom fields



Good indexing + ANN + Metadata filtering = Faster search + Better results + Lower cost





Indexing & Vector Databases (Part 2)

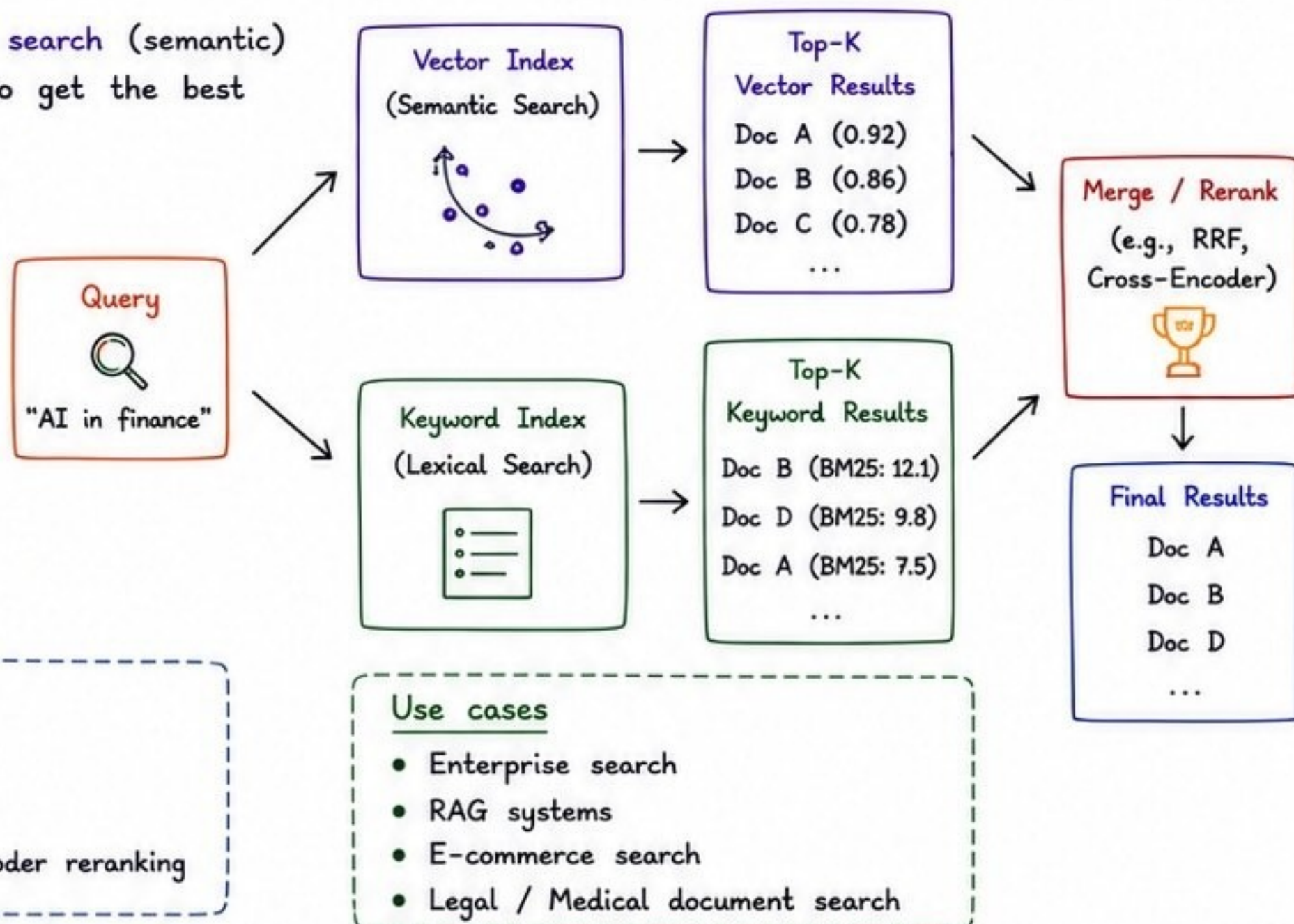


1. Hybrid indexing

Hybrid indexing combines vector search (semantic) with keyword search (lexical) to get the best of both worlds.

Why hybrid?

- Dense vectors capture meaning but may miss exact terms.
- Keyword search is great for exact matches & rare terms.
- Hybrid = better recall + better precision.



2. Popular vector databases

Different vector databases have different strengths. Choose based on your scale, use case & integration needs.

Feature	FAISS	Pinecone	Weaviate	Milvus	Qdrant
Type	Open Source (Library)	Managed Service (Cloud)	Open Source / Cloud	Open Source	Open Source / Cloud
Best For	Research, prototyping, high performance search	Production apps, easy to scale	Semantic search with metadata & AI features	Large-scale enterprise applications	Production apps with rich filtering & speed
Vector Index (ANN)	IVF, HNSW, Flat, PQ, OPQ	HNSW	HNSW	IVF, HNSW, DiskANN	HNSW
Metadata / Filtering	Limited (needs external handling)	Metadata filtering, namespaces	Powerful filters, GraphQL	Metadata filtering, scalar, range	Rich filtering, nested payloads
Scalability	Single machine (needs sharding manually)	Auto-scaling, serverless option	Horizontal scaling, Sharding	Distributed, horizontal scaling	Horizontal scaling, replication
Persistence	Local disk	Fully managed cloud	Local / Cloud	Local / Cloud	Local / Cloud
Query Types	Vector similarity (L2, IP)	Vector similarity + Metadata filter	Vector, BM25, Hybrid, Graph	Vector similarity + Metadata	Vector, Hybrid, Full-text, Filters
Language / SDKs	C++ (core), Python bindings	REST API, Python, JS, etc.	Python, JS, Go, REST, GraphQL	Python, Java, Go, REST	Python, JS, Go, Rust, REST
Cloud Availability	✗ (Self-hosted)	✓ Yes	✓ Yes (Weaviate Cloud)	✓ Yes (Zilliz Cloud)	✓ Yes (Qdrant Cloud)
Highlights	Highly optimized, great for research	Easy to use, managed, reliable	Graph + Vector + AI modules	High performance at scale	Fast, feature-rich, developer friendly

Quick Choosing Guide

- Just starting / research → **FAISS**
- Need managed cloud with less ops → **Pinecone**

- Need advanced features (Graph, Modules) → **Weaviate**
- Handling billions of vectors, self-hosted → **Milvus**
- Need fast filtering & hybrid search → **Qdrant**



Good indexing + Right Vector DB = Fast, Relevant, Scalable Search!





Retrieval (Part 1)



Retrieval is the process of finding the most relevant information (chunks/documents) from a knowledge base for a given query.

1. Semantic search (Vector Search)

Finds relevant documents based on meaning (semantic similarity) using embeddings.

How it works?

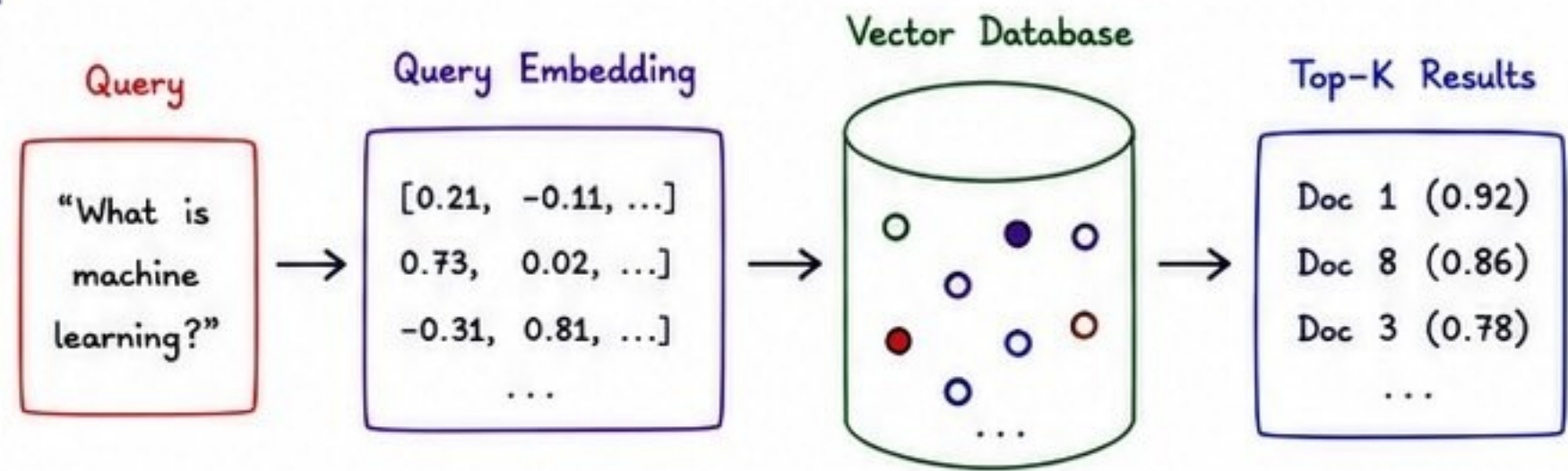
- 1 Convert query into embedding.
- 2 Compare with document embeddings using similarity metric.
- 3 Return top-k most similar chunks.

Key points

- Captures meaning, synonyms, context
- Works well for natural language queries
- Depends on embedding quality & metric

Common use cases

- Conceptual questions
- Research / Exploratory search
- RAG systems



2. Keyword / BM25 search

Finds relevant documents based on **exact word matching** and term importance.

How it works?

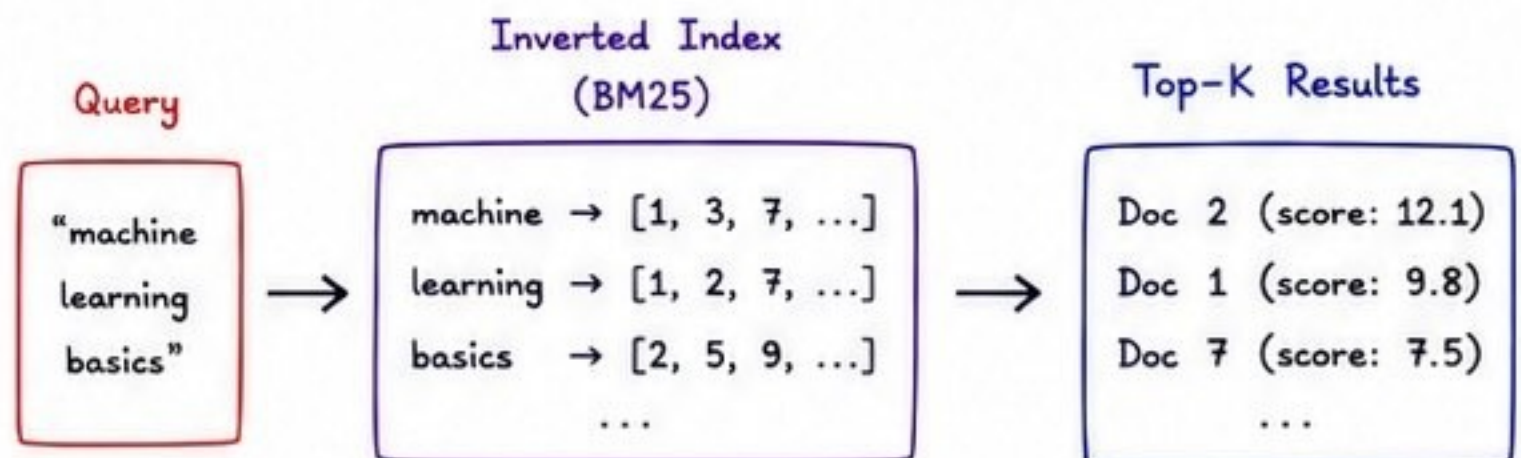
- 1 Tokenize the query.
- 2 Match terms in documents using BM25 scoring.
- 3 Return top-k documents with highest scores.

Key points

- Good for exact terms, names, codes, numbers
- Interpretable & fast
- Less effective for synonyms or paraphrasing

Common use cases

- FAQ search
- Legal / Policy documents
- Technical lookup (IDs, product codes, etc.)



3. Hybrid search

Combines **semantic search** (vector) and **keyword search** (BM25) to leverage the strengths of both.

How it works?

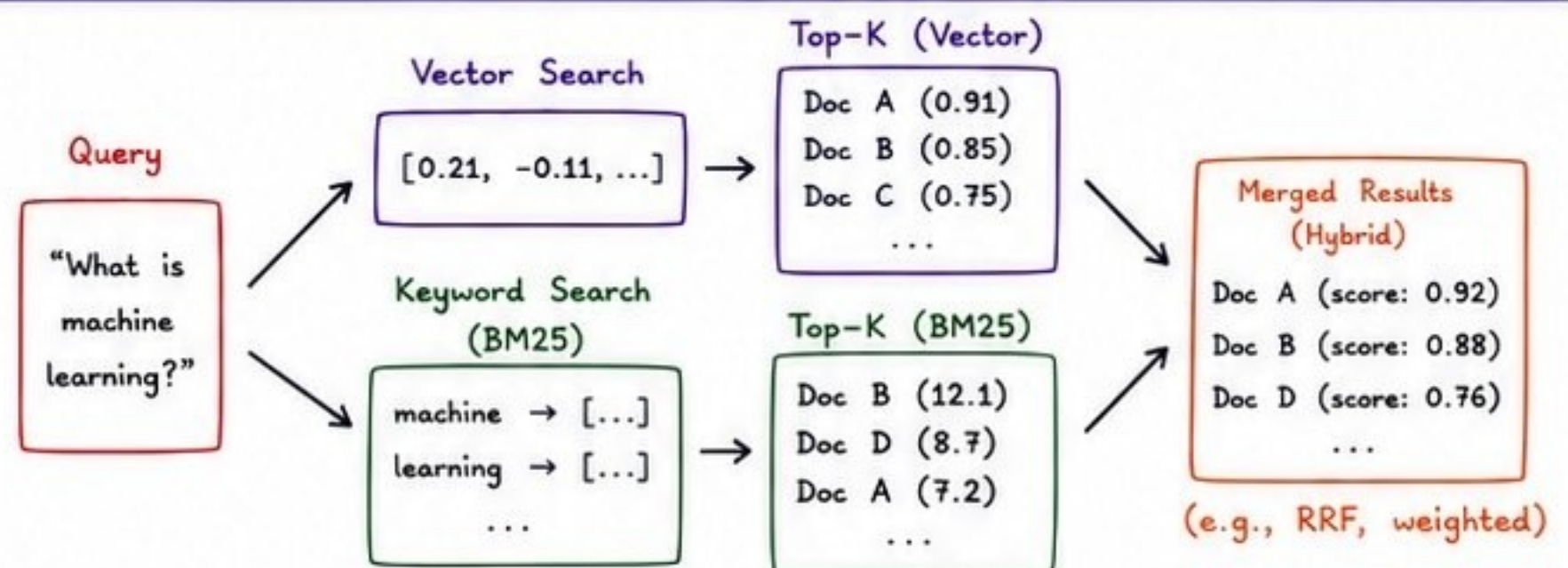
- 1 Run both semantic search and BM25 search.
- 2 Combine results using a ranking strategy (e.g., Reciprocal Rank Fusion, weighted sum).
- 3 Return top-k merged results.

Key points

- Better recall + precision
- Handles both exact match and semantic meaning
- Best general-purpose retrieval for RAG

Common use cases

- Enterprise search
- RAG (production systems)
- When you don't know whether users search by keywords or by meaning



Right Retrieval → Right Context → Better Answers from LLMs!





Retrieval (Part 2)



4. Query expansion

Improves recall by expanding the original query with synonyms, related terms or rephrased sentences.

How it works?

1. Take the original query.
2. Generate expanded terms or phrases (using LLM, thesaurus, or embeddings).
3. Search using the expanded query.
4. Return top-k results.

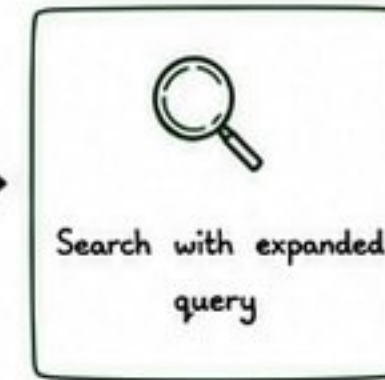
Original Query

"best practices for data security"

Expanded Query (examples)

- data security best practices
- recommended practices for data protection
- data security guidelines
- how to secure data
- information security tips
- ...

Vector / Keyword Search (Hybrid)



Top-K Results

Doc 3 (0.91)
Doc 8 (0.87)
Doc 1 (0.82)
...

Key points

- Increases recall, reduces missed results
- Helpful for short or vague queries
- Use carefully (avoid drift / noise)

Common use cases

- User queries with limited context
- Domain-specific search
- RAG systems to improve answer quality



5. Multi-query retrieval

Generate multiple diverse versions of a query and retrieve results for all to improve coverage and robustness.

How it works?

1. Generate multiple query variants.
2. Run retrieval for each variant.
3. Merge and rank results (deduplicate).
4. Return top-k results.

Original Query

"benefits of renewable energy"

Generated Query Variants (using LLM)

1. advantages of renewable energy
2. renewable energy benefits
3. why renewable energy is important
4. pros of clean energy
5. renewable energy advantages
- ...

Retrieve for each query variant

Results for Q1
Results for Q2
Results for Q3
Results for Q4
...

Merged Top-K Results

Doc 2 (0.93)
Doc 7 (0.89)
Doc 4 (0.86)
...

Key points

- Increases recall and coverage
- Reduces sensitivity to query wording
- More compute cost (multiple searches)

Common use cases

- Complex / broad topics
- User queries with unknown intent
- RAG for higher answer completeness

More queries
→ More chances to find the right docs!

6. Self / Corrective retrieval

The system evaluates retrieved results and iteratively refines the query or retrieval strategy to improve relevance.

How it works?

1. Retrieve initial results.
2. Evaluate relevance (LLM / heuristics).
3. If not satisfied, rewrite query / adjust filters / change strategy.
4. Repeat until results meet threshold or max attempts reached.

Retrieval Loop

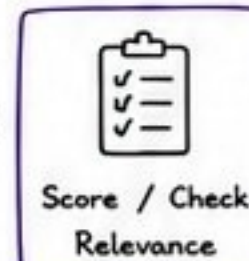
User Query

"causes of climate change"

Retrieve (Initial)



Evaluate (Relevance)



Satisfactory?
Yes
No

Final Results

Doc 1 (0.92)
Doc 4 (0.88)
Doc 7 (0.82)
...

Refine Query / Adjust Strategy

- Rewrite query
- Add missing context
- Change filters / use different search (e.g., hybrid)

Key points

- Improves precision and relevance
- Adapts based on feedback / evaluation
- Useful when initial results are weak or noisy

Common use cases

- Hard / ambiguous queries
- High-accuracy RAG systems
- When incorrect or partial answers are returned

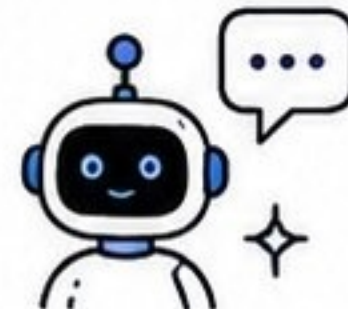


Better Retrieval = Better Context → Better Answers from LLMs!





Reranking



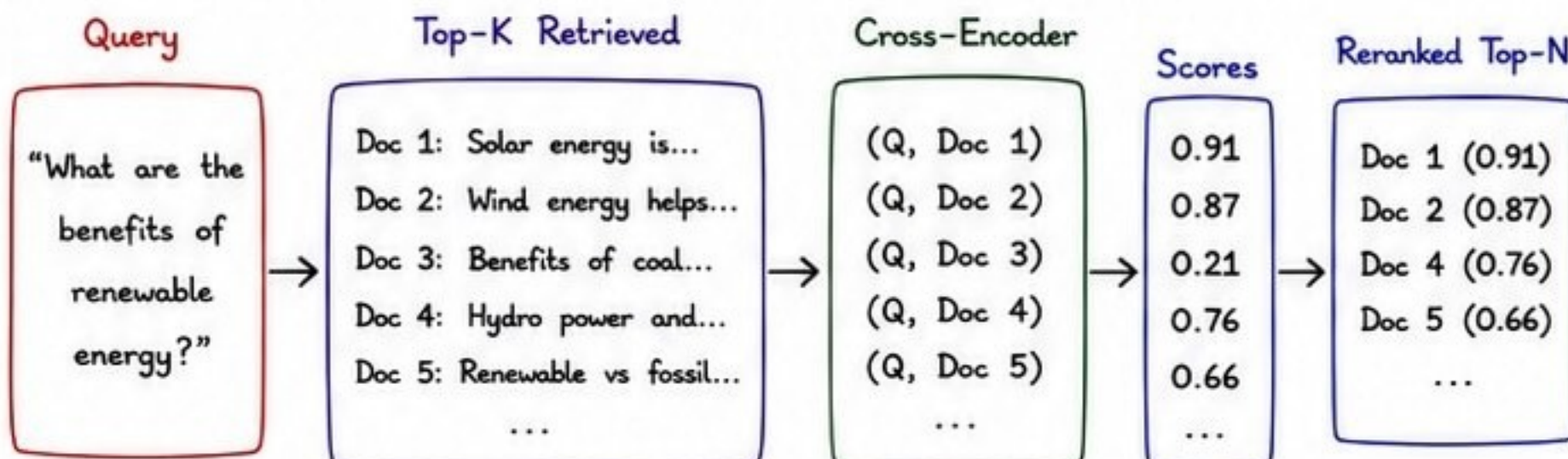
Reranking is the process of reordering initially retrieved documents using a more powerful model or method to improve relevance.

1. Cross-encoder reranking

Uses a cross-encoder model that takes the query and each document together as input to produce a relevance score.

How it works?

- 1 Retrieve Top-K documents using a fast retriever (BM25 / Vector).
- 2 For each doc, pass (query, document) pair to cross-encoder.
- 3 Get relevance scores.
- 4 Sort by score → return Top-N.



Key points

- High accuracy, understands deep semantics.
- More expensive (computationally).
- Best for Top-K → Top-N reranking.

Common use cases

- Improve precision in RAG.
- Enterprise search.
- When top results need to be highly relevant.

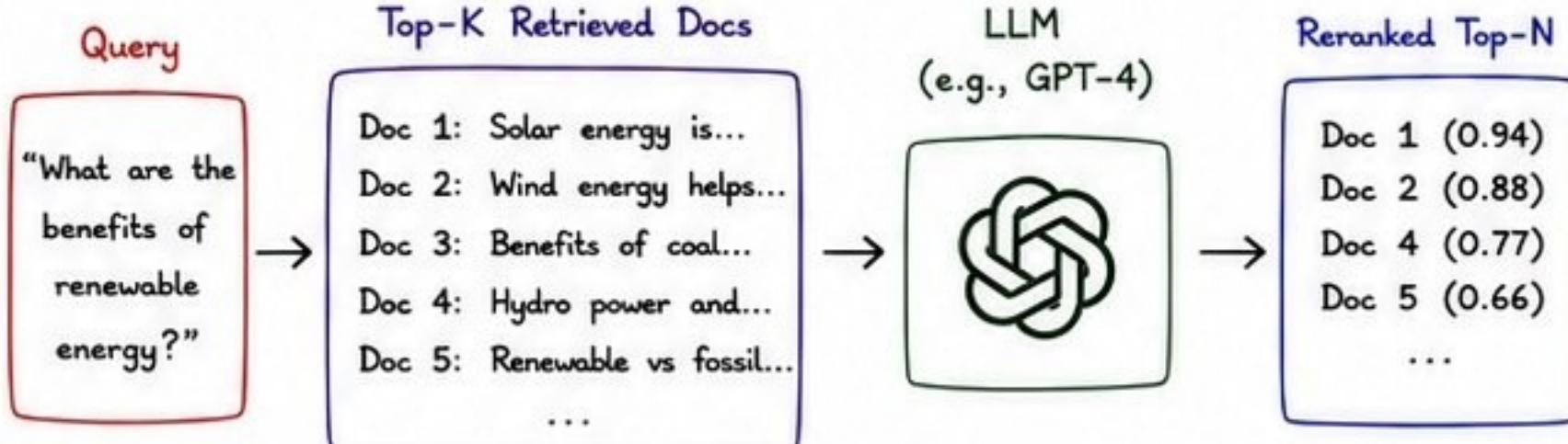


2. LLM reranking

Uses a large language model to judge which documents are most relevant to the query.

How it works?

- 1 Retrieve Top-K documents (BM25 / Vector).
- 2 Send query + documents to LLM.
- 3 LLM evaluates and ranks by relevance.
- 4 Return Top-N results.



Key points

- Flexible, can use reasoning & context.
- Can consider intent, nuances, and summaries.
- More costly and higher latency.

Common use cases

- Complex queries needing reasoning.
- Domain-specific or nuanced relevance.
- When cross-encoder is too expensive.

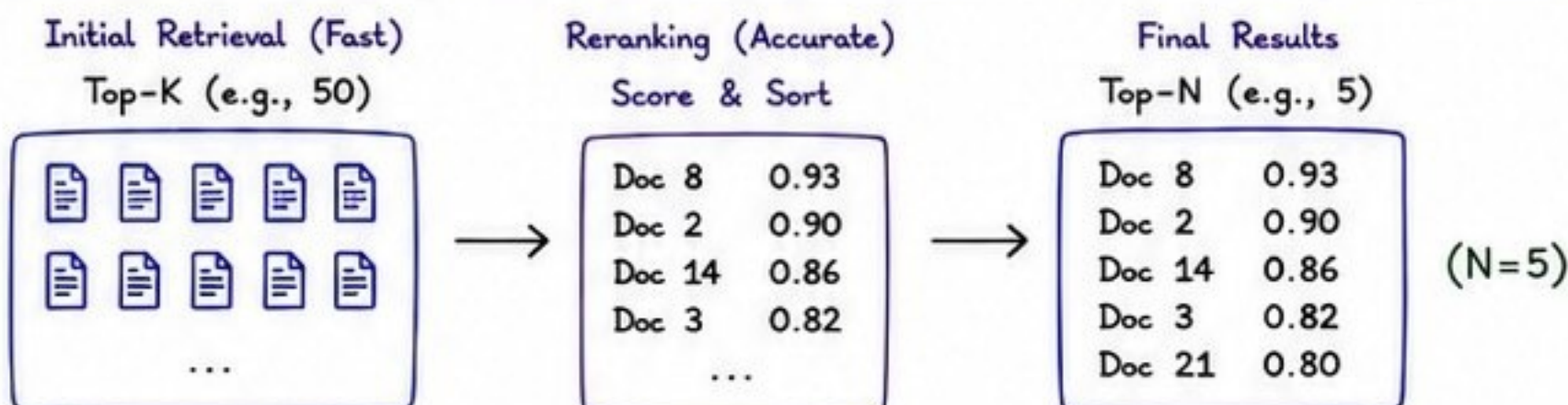


3. Top-K → Top-N retrieval

Retrieve a larger set of candidates (Top-K) and rerank to return a smaller, high-quality set (Top-N).

Why it matters?

- Initial retrieval may miss the best docs.
- Reranking needs a larger candidate pool.
- Balance between recall (K) and precision (N).



Tips

- Choose K big enough (20-100+) to get good candidates.
- Choose N based on use case (3-10 typically).

Rule of thumb

Higher K → better recall
Higher N → better user experience



4. Retrieval optimization

Techniques to make retrieval + reranking faster, cheaper, and more effective.

Optimization techniques

- Efficient indexing (HNSW, IVF, PQ).
- Adaptive K (increase K for hard queries).
- Early stopping in reranking.
- Caching frequent queries & results.
- Parallelization (batch reranking).
- Use filters to reduce search space.

Speed



- Use ANN (HNSW).
- Batch cross-encoder inference.
- Early exit / pruning.
- Quantization / Distillation.

Quality



- Better embeddings.
- Hybrid search.
- Cross-encoder or LLM reranking.
- Query expansion.

Cost



- Limit K and N wisely.
- Use lightweight models when possible.
- Cache & reuse results.
- Monitor & evaluate.

Good reranking gets candidates.
Good reranking gets the right ones to the top!





RAG Evaluation (Part 1)



RAG Evaluation measures how well the retrieval component finds the right information and how trustworthy the final answers are.

1. Retrieval evaluation

Retrieval evaluation focuses on how good the retriever is at finding relevant documents (chunks) for a query.

What do we evaluate?

- Are the relevant documents in the top results?
- Are irrelevant documents ranked low?
- Does the retriever return enough useful information for the LLM to answer?

What is needed?

- Test set of queries
- Ground truth relevant documents for each query
- Ranked list of retrieved documents

Query

"What are the benefits of solar energy?"

Retriever
(BM25 / Vector / Hybrid)



Top-K Retrieved Documents



Evaluate Ranking Quality



Why it matters?

- Good retrieval → better context for LLM
- Poor retrieval → LLM can't answer even with a strong model
- Retrieval quality upper-bounds RAG performance



2. Precision / Recall / MRR / NDCG

Common metrics used to evaluate retrieval quality.

Precision@k

Of the top-k retrieved documents, how many are relevant?

$$\text{Precision@k} = \frac{\# \text{ Relevant in Top-k}}{k}$$

Range: 0 to 1 (Higher is better)

Example:

Top-5 results = [R, I, R, I, R]
Relevant in Top-5 = 3
Precision@5 = 3/5 = 0.60

Recall@k

Of all relevant documents, how many did we retrieve in top-k?

$$\text{Recall@k} = \frac{\# \text{ Relevant in Top-k}}{\text{Total \# Relevant}}$$

Range: 0 to 1 (Higher is better)

Example:

Total relevant docs = 10
Relevant in Top-5 = 3
Recall@5 = 3/10 = 0.30

MRR

(Mean Reciprocal Rank)

Considers the rank of the first relevant document.

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i}$$

rank_i = rank of first relevant doc for query i

Range: 0 to 1 (Higher is better)

Example:

First relevant at rank 3 → 1/3
First relevant at rank 1 → 1

NDCG@k

(Normalized Discounted Cumulative Gain)

Considers the rank and relevance scores of all documents.

$$\text{DCG@k} = \sum_{i=1}^k \frac{\text{rel}_i}{\log_2(i+1)}$$

$$\text{NDCG@k} = \text{DCG@k} / \text{IDCG@k}$$

Range: 0 to 1 (Higher is better)

Example:

Higher when relevant docs are ranked at top.

Quick Comparison

Metric

Precision@k

Recall@k

MRR

NDCG@k

Focus

Top results quality

Coverage of all relevant

Rank of first relevant doc

Overall ranking quality

Best For

When you care about exact results in top-k

When missing relevant docs is costly

When first correct hit is most important

When rank and relevance both matter

Tip:

Use a combination of metrics. No single metric tells the whole story!



3. Faithfulness

Faithfulness (or Groundedness) checks whether the answer is supported by the retrieved context.

What do we check?

- Does the answer use only information from the retrieved documents?
- Does it avoid hallucinations or adding extra facts?
- Is every important claim supported by the context?

Example

Retrieved Context

"Solar energy reduces electricity bills and lowers carbon emissions."

Generated Answer

"Solar energy reduces electricity bills and helps reduce carbon emissions."

Faithfulness Check

Supported by context

How to evaluate?

- LLM-as-a-judge (prompt: is answer supported by context? Yes/No)
- Claim-level verification
- Human evaluation

Why it matters?

- Users trust answers that are grounded.
- Reduces hallucinations.
- Critical for enterprise/production systems.



4. End-to-end optimization

Techniques to make retrieval + reranking faster, cheaper, and more effective.

Optimization techniques

- Efficient indexing (HNSW, IVF, PQ).
- Adaptive K (increase K for hard queries).
- Early stopping in reranking.
- Caching frequent queries & results.
- Parallelization (batch reranking).
- Use filters to reduce search space.

RAG Pipeline

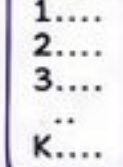
Query



Retriever (Top-K)



Rerank



Generate Answer



Final Answer



End-to-end Scores

- Overall Answer Relevancy
- Faithfulness (Groundedness)
- Answer Correctness
- User Satisfaction

Why it matters?

- Looks at the whole system, not just one part
- Catch issues that component metrics miss
- Drives continuous improvement



Measure → Analyze → Improve → Deliver Better Answers!





RAG Evaluation (Part 2)



4. Answer relevance

Measures how well the final answer addresses the user query.

What do we check?

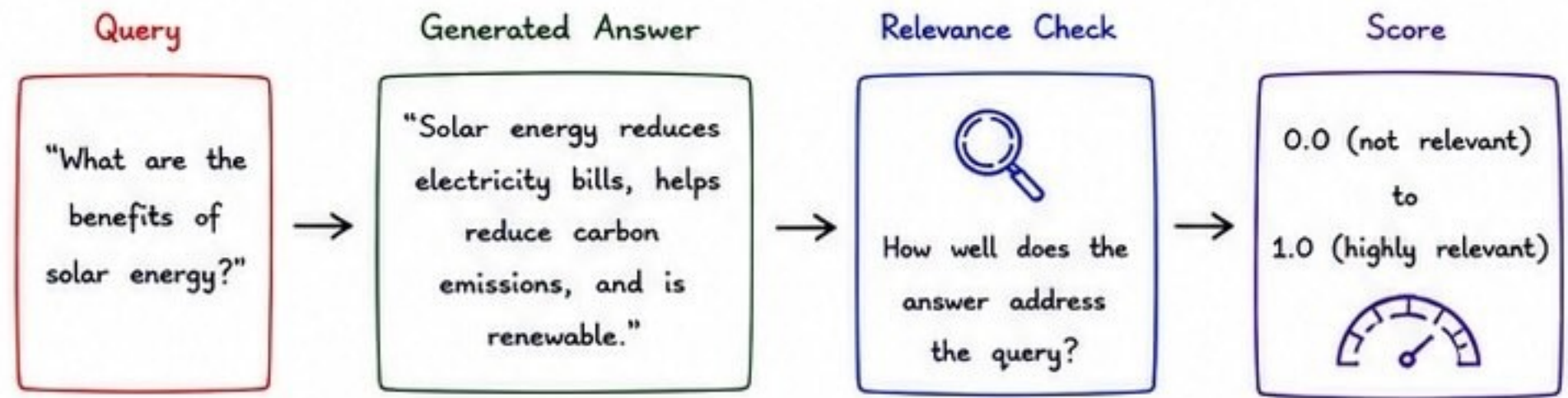
- Does the answer actually answer the question?
- Is it complete, specific and on-topic?
- Would the user be satisfied with this answer?

How to evaluate?

- LLM-as-a-judge (prompt: Does the answer answer the question? Rate 0-1)
- Human evaluation

Why it matters?

- High answer relevance → better user satisfaction
- Low relevance → even good context won't help
- Key for production RAG systems



5. Context relevance

Measures how relevant the retrieved context is to the user query.

What do we check?

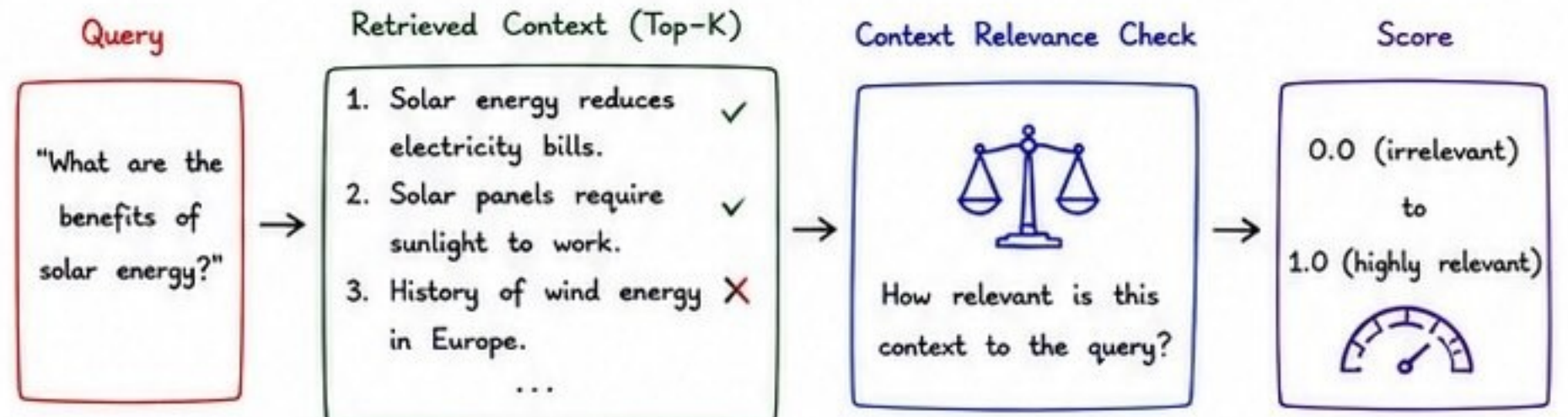
- Is the retrieved context about the right topic?
- Does it contain the information needed to answer?
- Is there too much irrelevant context (noise)?

How to evaluate?

- LLM-as-a-judge (rate relevance of each chunk)
- Overlap with ground-truth context
- Embedding similarity (query ↔ context)

Why it matters?

- Irrelevant context → misleading or wrong answers
- Relevant context → higher faithfulness & accuracy
- Reduces noise for the LLM



6. RAGAS / DeepEval

Tools/frameworks for evaluating RAG systems automatically.

RAGAS (open-source)

- Python library for RAG evaluation
- Built-in metrics:
 - Context Precision
 - Context Recall
 - Faithfulness
 - Answer Relevancy
 - Answer Correctness
- Works with LLM-as-a-judge
- Supports batch evaluation and reports

DeepEval (LLM Evaluation Platform)

- General LLM evaluation platform
- Supports RAG-specific metrics:
 - Context Relevancy
 - Answer Relevancy
 - Faithfulness
 - Hallucination
 - Bias, Toxicity, etc.
 - ...
- Custom test cases & datasets
- CI/CD integration & dashboards

Example Usage (RAGAS)

```

from ragas import evaluate
from ragas.metrics import (
    context_precision, context_recall,
    faithfulness, answer_relevancy
)

result = evaluate(
    dataset=my_dataset,
    metrics=[context_precision,
             context_recall, faithfulness,
             answer_relevancy]
)

result # returns scores and reports
  
```

RAGAS



DeepEval



These tools save time, standardize metrics, and help track improvements over time!

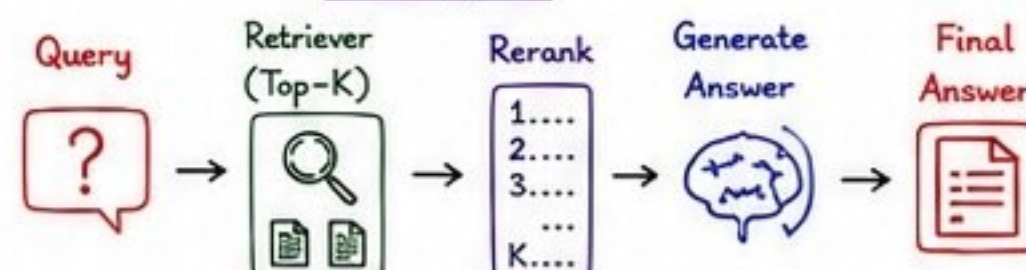
7. End-to-end evaluation

Evaluates the entire RAG pipeline from retrieval to final answer.

What do we measure?

- Retrieval quality
- Context relevance
- Answer generation
- Final answer relevance & faithfulness
- Overall user satisfaction

RAG Pipeline



End-to-end Scores

- Overall Answer Relevancy
- Faithfulness (Groundedness)
- Answer Correctness
- User Satisfaction

Why it matters?

- Looks at the whole system, not just one part
- Catch issues that component metrics miss
- Drives continuous improvement



Measure → Analyze → Improve → Deliver Better Answers!





Advanced RAG



Advanced RAG techniques combine structured data, relationships, and databases with retrieval to deliver more accurate and contextual answers.

1. Graph RAG

Uses graph-based retrieval strategies to capture relationships between entities and improve context.

How it works?

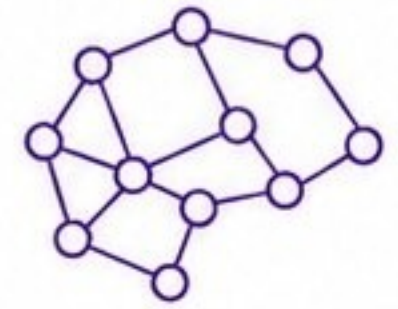
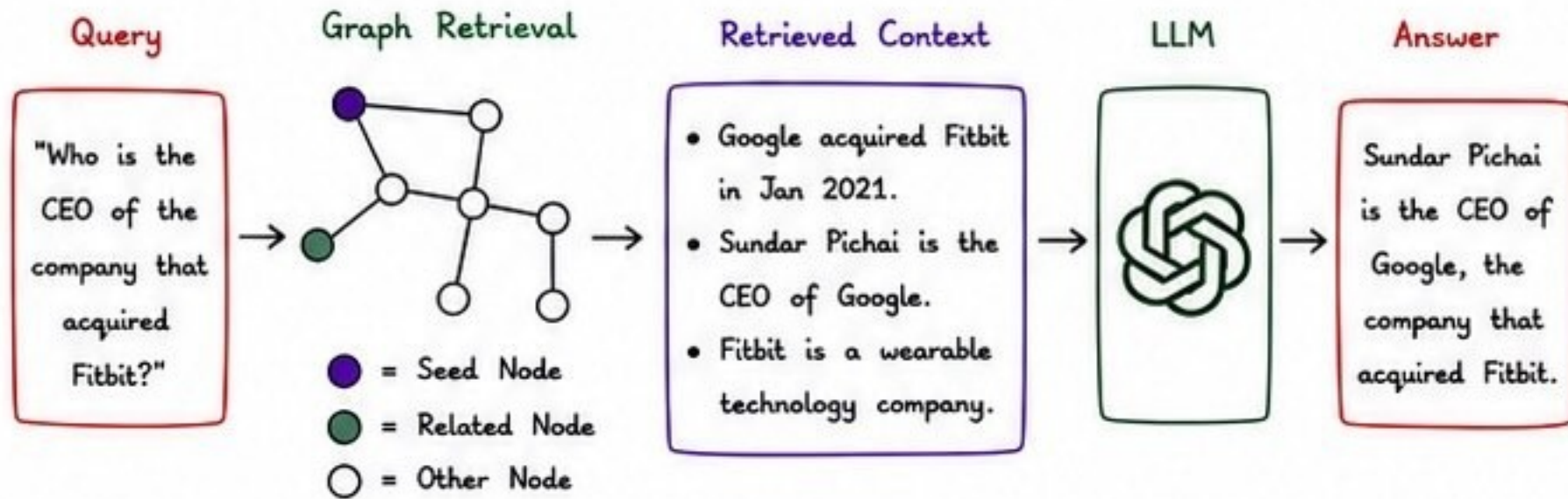
- ① Convert documents into graph (entities + relations).
- ② For a query, find relevant nodes (seed nodes).
- ③ Expand to related nodes (k-hop traversal).
- ④ Retrieve content from relevant nodes/edges.
- ⑤ Pass retrieved context to LLM for answer generation.

Key points

- Captures relationships & dependencies
- Better for complex, multi-hop questions
- Reduces missing context problem

Use cases

- Fraud detection, compliance, investigation
- Recommendation systems
- Research & knowledge discovery



2. Knowledge Graph + RAG

Combines the power of Knowledge Graphs (KGs) with RAG to provide structured and factual answers.

How it works?

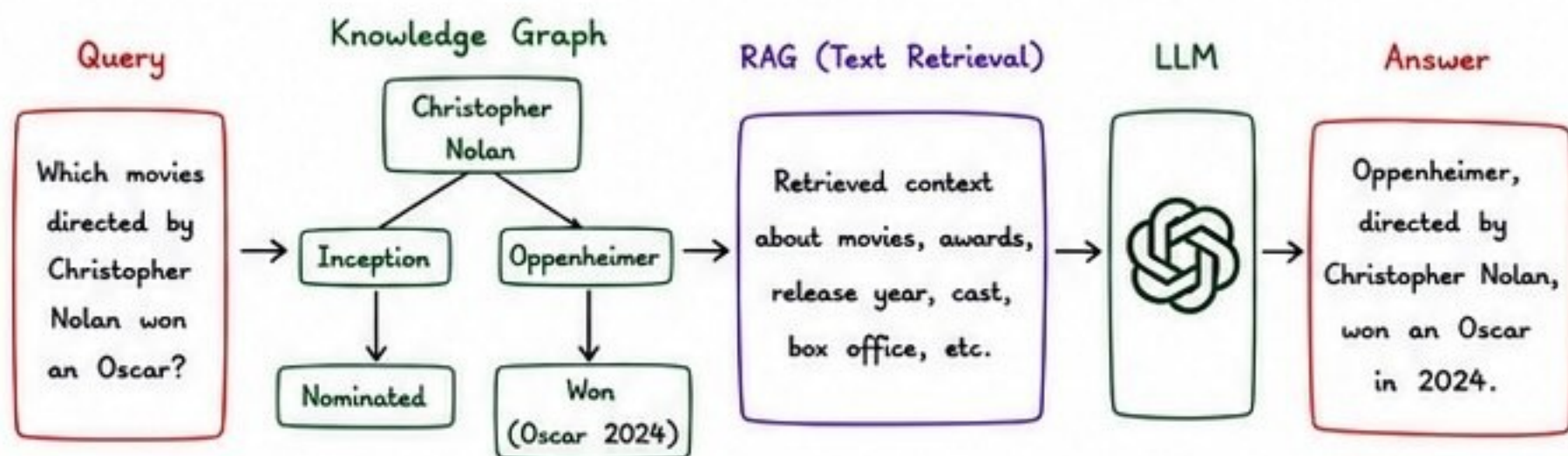
- ① Build or use an existing Knowledge Graph (KG).
- ② Use KG to understand entities & relationships.
- ③ Retrieve unstructured data (text) via RAG.
- ④ Fuse KG insights + retrieved context.
- ⑤ Generate answer using LLM.

Key points

- KG gives structured, reliable facts
- RAG provides rich unstructured details
- Improves accuracy & reduces hallucinations

Use cases

- Enterprise search
- Customer support
- Domain-specific Q&A (healthcare, legal, finance)



3. SQL + RAG

Combines SQL (for structured data) with RAG (for unstructured data) to answer complex questions that need both.

How it works?

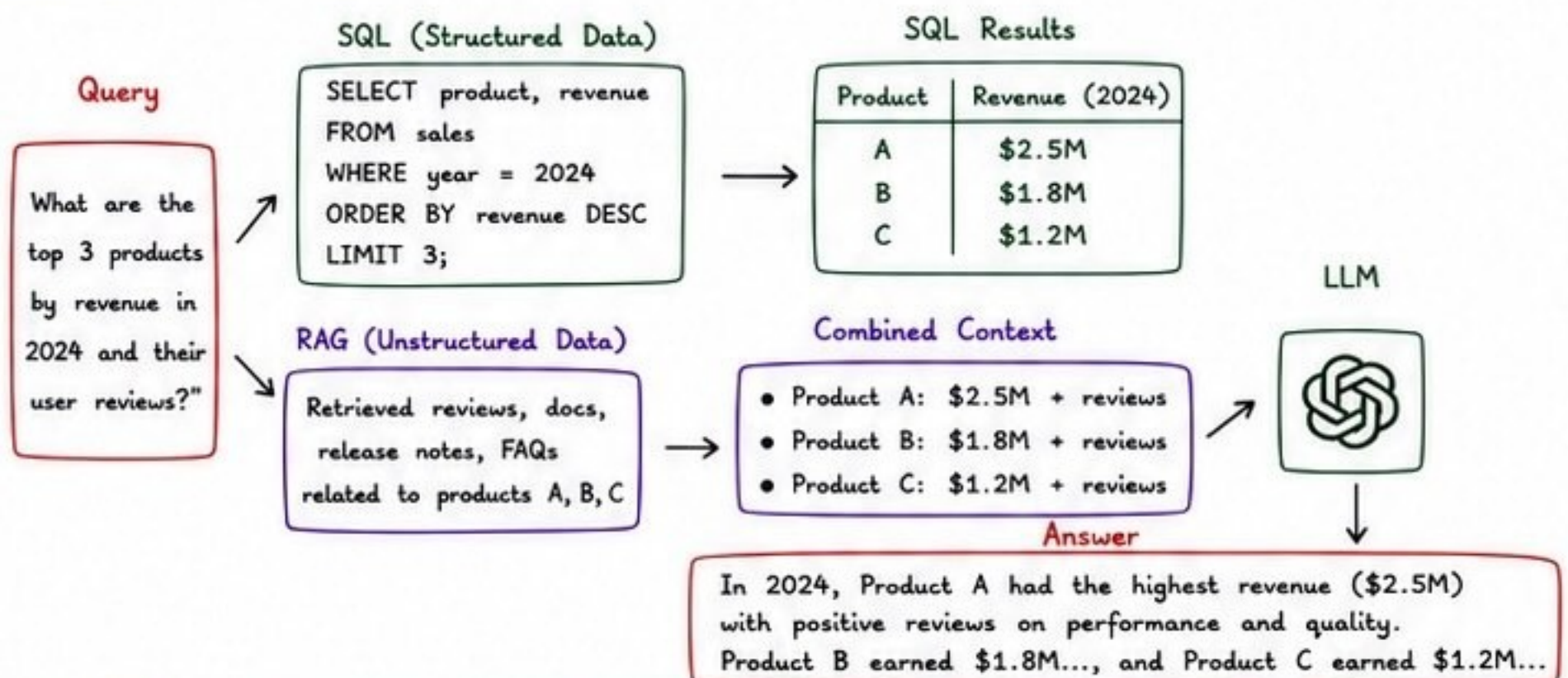
- ① Understand the query and decide SQL or RAG or both.
- ② Generate SQL query (if needed) and fetch results from database.
- ③ Retrieve relevant documents via RAG.
- ④ Combine SQL results + retrieved context.
- ⑤ Generate final answer using LLM.

Key points

- Best of both worlds: structured + unstructured
- Handles analytical + semantic questions
- Requires query routing & orchestration

Use cases

- Business intelligence + research
- E-commerce analytics
- Customer feedback analysis



Advanced RAG = Better Context + Deeper Insights + Reliable Answers





Advanced RAG (Part 2)

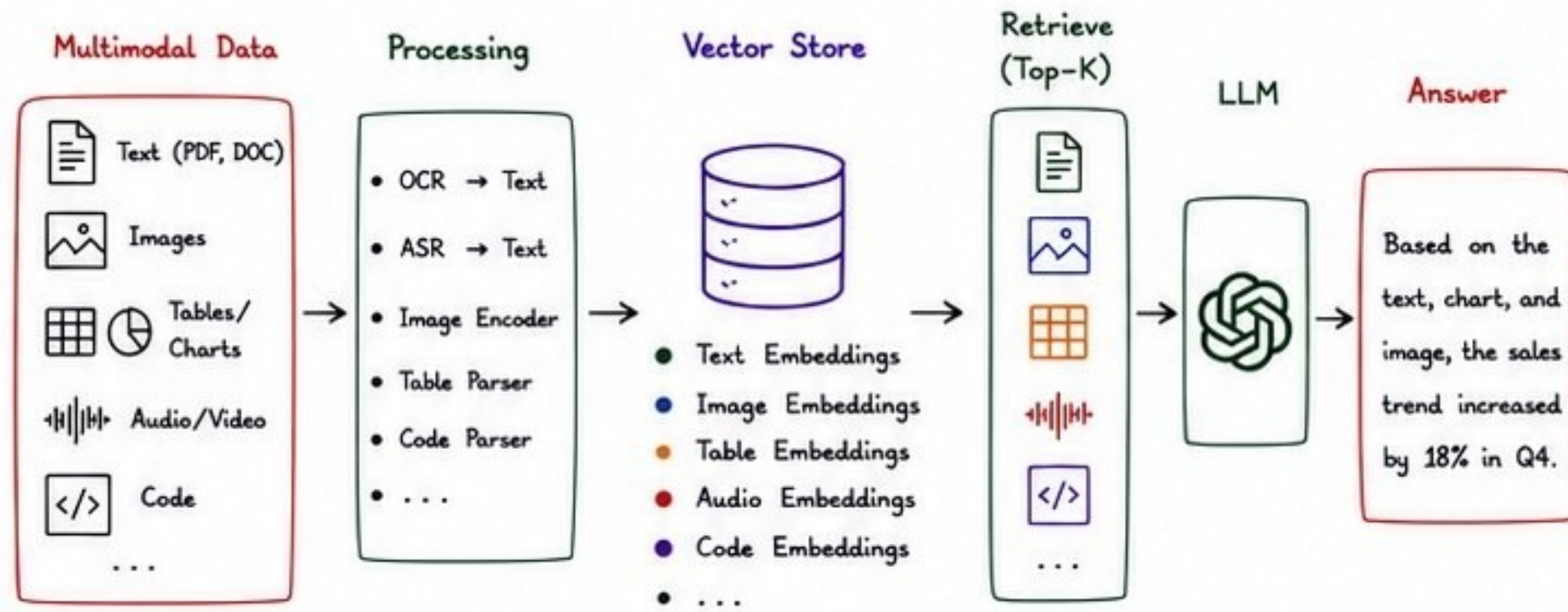


4. Multimodal RAG

Retrieves and reasons over multiple data types like text, images, tables, charts, audio, code, etc.

How it works?

- 1 Ingest multimodal documents.
- 2 Extract text (OCR/ASR), create embeddings for each modality.
- 3 Store in a vector store with modality tags.
- 4 Retrieve relevant chunks across modalities.
- 5 Pass retrieved context to LLM (with modality information) to answer.



Key points

- Understands and retrieves across modalities
- Captures richer context
- Needs modality-aware models/embeddings

Use cases

- Research (papers with figures & tables)
- Finance (charts + reports)
- Support (screenshots, logs, voice notes)

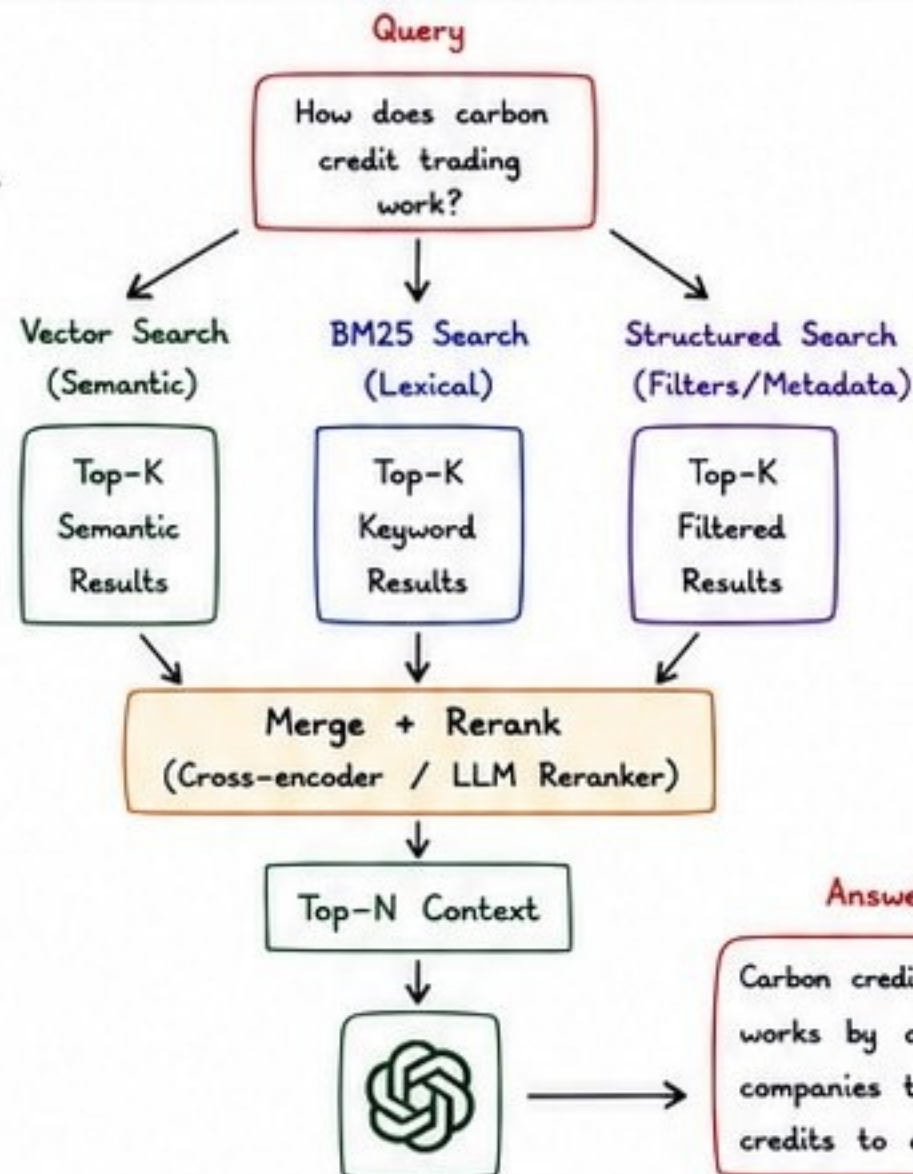


5. Hybrid RAG

Combines multiple retrieval techniques to get the best of semantic + lexical + structured search.

How it works?

- 1 Convert query → embeddings (semantic)
- 2 Perform vector search (semantic retrieval)
- 3 Perform keyword/BM25 search (lexical retrieval)
- 4 (Optional) Structured search (filters/metadata)
- 5 Merge & rank results (Reranker / LLM)
- 6 Pass top results to LLM.

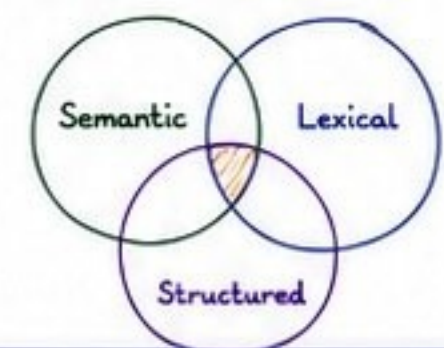


Why Hybrid?

- Semantic search finds meaning
- Lexical search finds exact matches
- Structured search applies business rules
- Reranking improves final relevance

Use cases

- Enterprise search
- E-commerce (products + filters)
- Legal / compliance
- Technical support

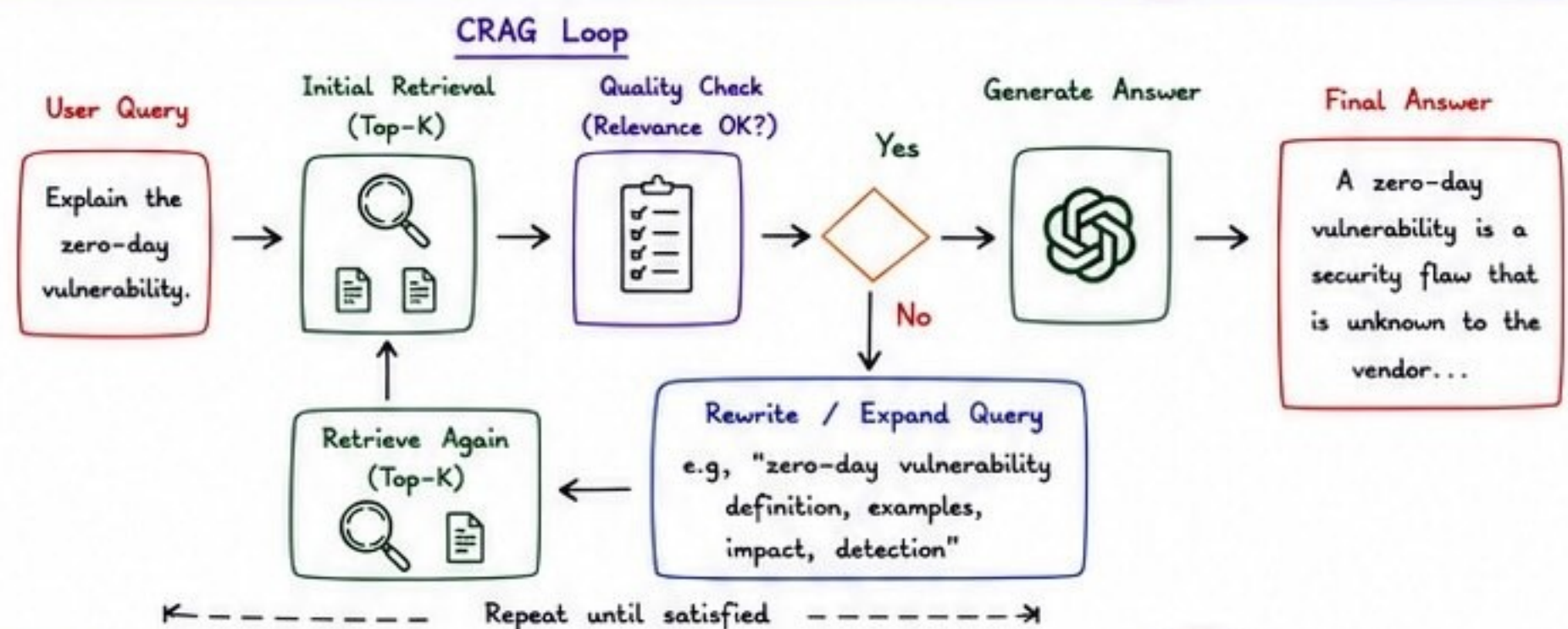


6. Corrective RAG (CRAG)

Detects when retrieval is insufficient or wrong, and corrects itself by refining the query, retrieving again, and improving the answer.

How it works?

- 1 Retrieve initial results.
- 2 Check retrieval quality (relevance/coverage).
- 3 If not satisfactory → rewrite/expand query.
- 4 Retrieve again.
- 5 Iterate until good enough or max attempts.
- 6 Generate final answer.

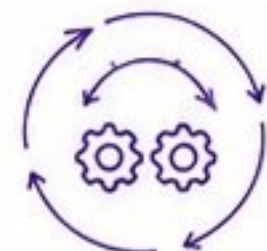


Key points

- Self-detects bad/insufficient retrieval
- Refines query automatically
- Improves factuality & completeness

Use cases

- Complex / ambiguous questions
- Technical topics
- Safety-critical applications



Advanced RAG Techniques → More Context → Better Answers!





Advanced RAG (Part 3)



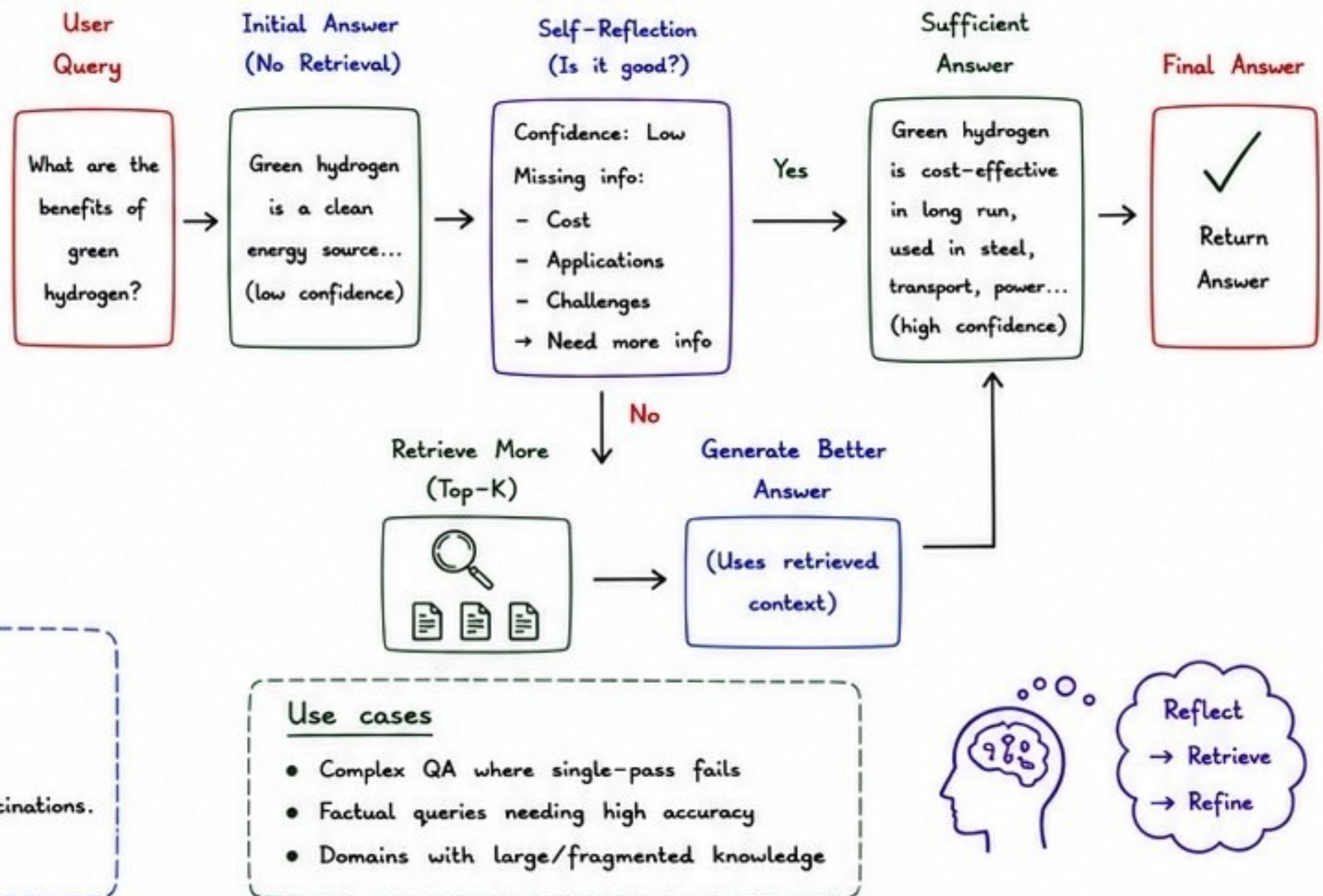
7. Self-RAG

LLM decides when and how to retrieve, what to retrieve, and whether the answer is good enough using self-reflection.

How it works?

- ① LLM looks at the question and generates an initial answer (may or may not retrieve).
- ② It reflects: Is the answer confident and well-supported?
- ③ If not, it retrieves more info and tries again.
- ④ Repeat until satisfied or max attempts.
- ⑤ Return final answer with confidence.

Self-RAG Loop



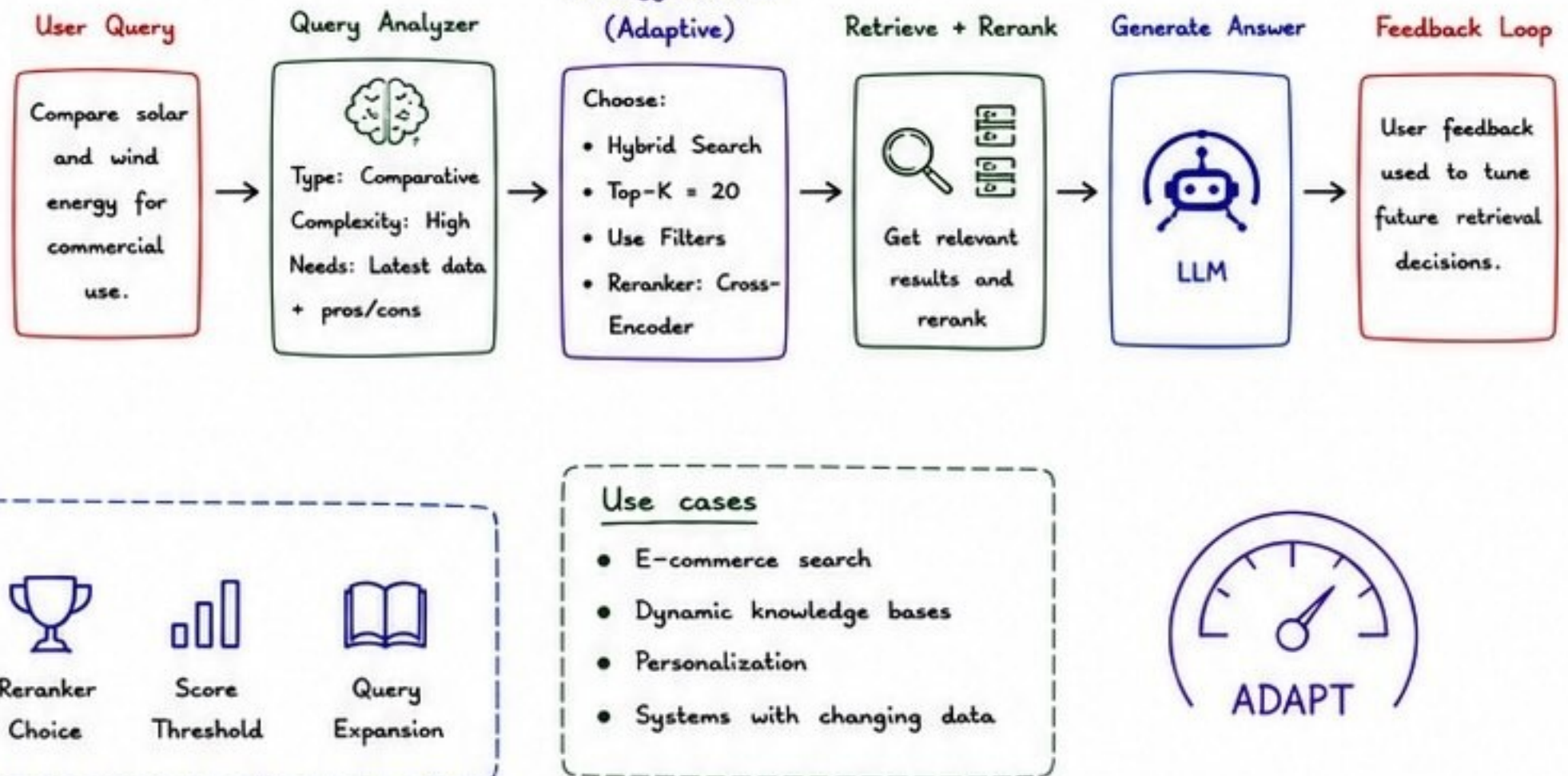
8. Adaptive RAG

System adapts the retrieval strategy based on the query type, complexity, or past feedback to get the best results.

How it works?

- ① Analyze the query (type, intent, complexity).
- ② Choose the best retrieval strategy (e.g., semantic, keyword, hybrid, filters).
- ③ Adjust Top-K, reranker, and other params.
- ④ Retrieve, rerank, and generate answer.
- ⑤ Learn from feedback and improve over time.

Strategy Selector (Adaptive)



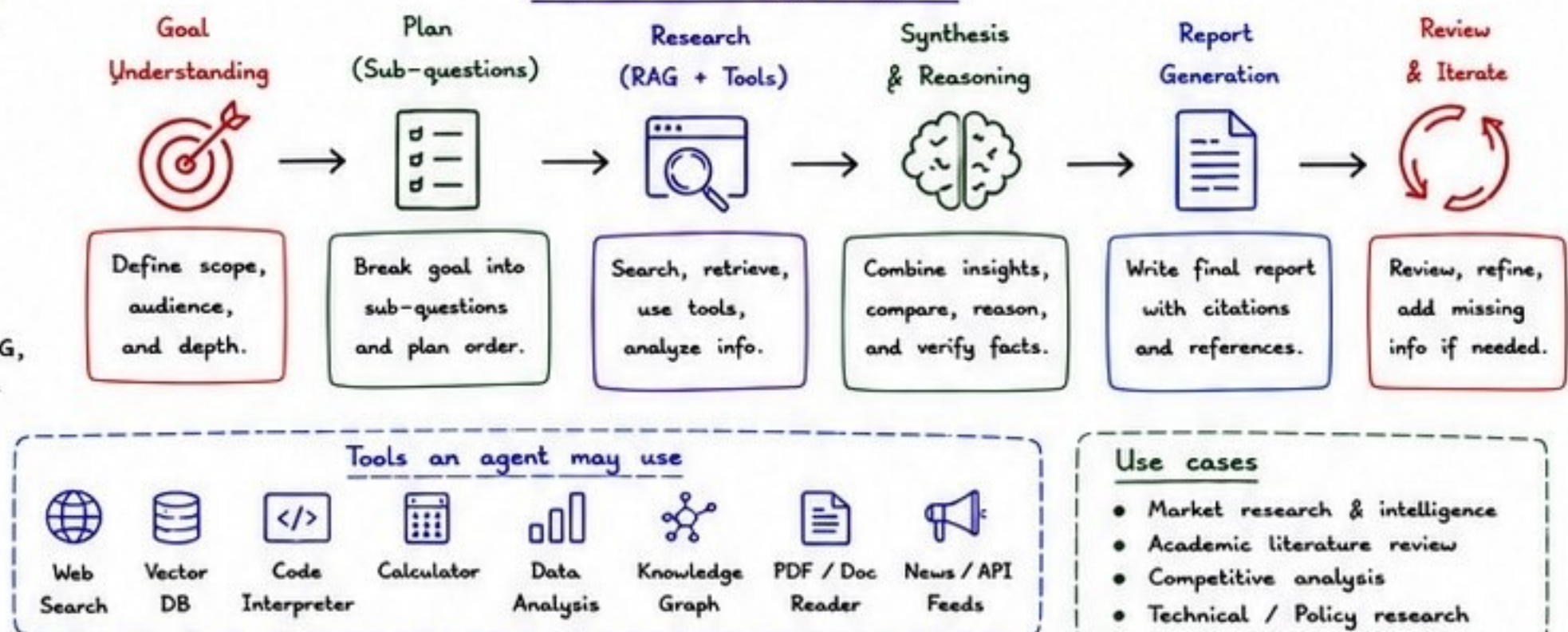
9. Deep Research Agents

Autonomous agents that perform deep, multi-step research using RAG, tools, and reasoning to deliver comprehensive reports.

How it works?

- ① Understand the research goal.
- ② Break down into sub-questions.
- ③ Search, retrieve, and analyze using RAG, use tools (web search, APIs, databases, calculators, code, etc.).
- ④ Synthesize findings and cite sources.
- ⑤ Iterate until the report is complete.

Deep Research Agent Pipeline



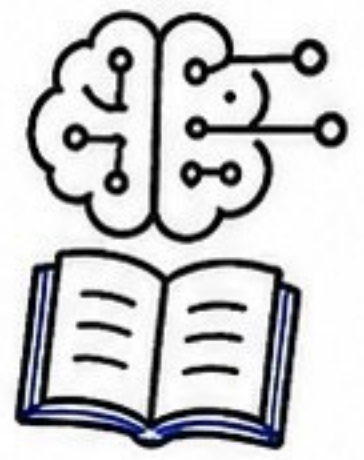
Smarter Retrieval + Adaptive Strategies + Autonomous Agents = Powerful RAG





30 Most Frequently Asked Interview Questions on RAG

(Intermediate to Advanced)



- ① What is RAG? How does it work?
- ② What are the key **components** of a RAG system?
- ③ What are the **advantages** of RAG over **fine-tuning** an LLM?
- ④ What are the **use cases** of RAG?
- ⑤ Explain the **RAG pipeline**.
- ⑥ What is the **difference** between RAG and traditional LLM?
- ⑦ How do you **ingest data** for RAG?
- ⑧ What are **chunking strategies**? How do you decide chunk size?
- ⑨ What are **embeddings**? Which embedding models have you used and why?
- ⑩ How do you **store and index** documents for RAG?
- ⑪ What is the difference between **dense retrieval** and **sparse retrieval**?
- ⑫ What is **hybrid search** in RAG?
- ⑬ What is **query expansion**? How is it useful in RAG?
- ⑭ What is **multi-query retrieval**?
- ⑮ What is **self retrieval / self query** in RAG?
- ⑯ What is **reranking** in RAG? Why is it important?
- ⑰ What is the difference between **bi-encoder** and **cross-encoder** rerankers?
- ⑱ How does **LLM reranking** work?
- ⑲ What is **Top-K retrieval**? How do you choose K?
- ⑳ How do you **evaluate retrieval** quality?
- ㉑ What are **Precision**, **Recall**, **MRR**, and **NDCG** in the context of RAG?
- ㉒ How do you **evaluate the generated answer** in RAG?
- ㉓ What is **faithfulness** in RAG? How do you measure it?
- ㉔ What are **RAGAS metrics**? How do you use them?
- ㉕ What is the difference between **RAGAS** and **DeepEval**?
- ㉖ What is **Self-RAG**? How does it improve RAG performance?
- ㉗ What is **Corrective RAG (CRAG)**?
- ㉘ What is **Adaptive RAG (ARAG)**?
- ㉙ What are common **challenges** in RAG and how would you solve them?
- ㉚ How would you design a **production-ready RAG system**?

• Topics Covered •



- ✓ RAG Basics & Components
- ✓ Data Ingestion & Indexing
- ✓ Retrieval Techniques
- ✓ Reranking
- ✓ Evaluation

- ✓ Advanced RAG (Self-RAG, CRAG, ARAG)
- ✓ Evaluation Frameworks (RAGAS, DeepEval)
- ✓ Challenges & Best Practices
- ✓ Production Considerations

